

RESEARCH ARTICLE

LLM-Assisted Reviewer Assignment via Auditable Expertise Matching

FARID BAGHERI¹, DAVIDE BUSCALDI^{ID}², AND DIEGO REFORGIATO RECUPERO^{ID}¹¹Department of Mathematics and Computer Science, University of Cagliari, 09121 Cagliari, Italy²Laboratoire d'Informatique de Paris Nord, Sorbonne Paris Nord University, Villetaneuse, 93430 Paris, France

Corresponding author: Diego Reforgiato Recupero (diego.reforgiato@unica.it)

ABSTRACT Accurate reviewer assignment is essential for scalable peer review, yet practical systems must balance semantic match quality with transparency, auditability, and manageable reviewer workloads. We study an end-to-end, LLM-assisted reviewer–paper matching pipeline in which large language models are used primarily for affinity construction by extracting and normalizing high-salience topical key phrases from submissions and reviewer publication summaries. We implement an agentic ensemble of heterogeneous producer models to generate candidate keyword sets, followed by a stronger judge agent that selects the best candidate for each input and representation, yielding compact, machine-parseable keyword profiles. Using a dataset of 663 submissions from 85 computer-science venues and 524 reviewer profiles, we evaluate three representations (titles, abstract summaries, and full abstracts) and compare lexical, neural, and graph-based affinity estimators. These include TF–IDF and Jaccard scoring (and their mean), Sentence-BERT similarity, keyword-aware retrieval with two-stage bi-encoder/cross-encoder re-ranking, and graph-based matchers over knowledge graphs and document-local graphs using Node2Vec embeddings with Jaccard-style fusion. We report MRR@3, Precision@3, and MAP@3. In addition, we simulate conference-level reviewer assignment to evaluate workload realism and inequality, summarizing reviewer loads both per-conference (macro) and globally (pooled), and quantifying inequality with the Gini coefficient. Results highlight consistent trade-offs across representations and matchers: two-stage re-ranking improves early-rank performance; keyword-aware Sentence-BERT increases top-3 concentration; and KG edge overlap is competitive on full abstracts, while some graph variants substantially concentrate reviewer workloads.

INDEX TERMS Large language model, LLM agent, recommender systems, reviewer assignment, semantic web.

I. INTRODUCTION

The reviewer assignment problem (RAP) concerns matching submitted papers with suitably qualified reviewers while satisfying practical constraints such as expertise coverage, capacity limits, conflicts of interest, and topic balance. Surveys from information retrieval, recommender systems, and operations research perspectives emphasize the trade-offs that arise at scale in scientific venues [1], [2]. Empirical evidence underscores the importance of assignment quality: in the NeurIPS 2021 repeat-committee experiment, independent program committees reached different decisions

for roughly 23% of submissions, suggesting that reviewer composition and assignment quality can materially affect outcomes [3]. In response, fairness-aware formulations have become increasingly prominent, including max–min objectives that ensure strong reviewer support even for the worst-served paper and mechanisms that promote balanced workloads across reviewers [4], [5].

Operationally, most automated approaches cast RAP as an optimization problem over paper–reviewer affinity scores, using maximum-weight bipartite matching or integer programming under capacity and conflict constraints. While exact solvers such as the Hungarian algorithm provide principled solutions under basic assumptions [6], the meaning of any global optimum is ultimately determined by the

The associate editor coordinating the review of this manuscript and approving it for publication was Wei Wei^{ID}.

quality of the underlying affinity scores. In practice, affinity construction is therefore the central challenge, rather than a purely downstream optimization detail.

Early affinity estimators relied on TF-IDF representations and cosine similarity over titles and abstracts, sometimes extended with topic modeling techniques such as LSI and LDA to improve semantic coverage and provide interpretable topic-level controls [7], [8], [9]. The Toronto Paper Matching System (TPMS) operationalized such pipelines at conference scale and remains influential in practice [7]. Although transparent and efficient, these representations can struggle with fine-grained scientific semantics, cross-domain terminology, and synonymy.

Contextual encoders such as BERT and domain-adapted variants such as SciBERT substantially improved similarity estimation for scholarly text [10], [11], [12]. Subsequent work incorporated richer signals, including co-authorship and citation graphs, venue recency, and subarea coverage into multi-factor objectives. More recently, instruction-tuned Large Language Models (LLMs) have been explored for generating structured representations or intermediate signals that can support reviewer matching [13]. In parallel, broader agentic systems have been proposed to support multiple stages of the peer-review workflow [14], [15], [16], [17].

However, integrating LLMs into reviewer matching also introduces important concerns. LLMs may produce fluent but incorrect content, exhibit hallucination behavior, encode societal biases, and be susceptible to prompt injection or adversarial manipulation [18], [19], [20], [21]. These risks have practical consequences: many venues restrict or prohibit generative AI in peer review to preserve confidentiality, provenance, and reliability. For example, a study of the top 100 medical journals reports that 91% prohibit uploading manuscript content to AI tools and 59% explicitly prohibit AI use in peer review [22].

Against this backdrop, we adopt a pragmatic middle-ground approach. We use LLMs to support the *affinity-building* stage without allowing generative models to replace human judgment or become opaque decision-makers. An LLM-based agent extracts salient keyphrases from titles, abstracts, and reviewer publication summaries, normalizes terminology such as acronyms and synonyms, and produces compact keyword profiles that can be inspected, stored, and audited. The LLM component is therefore used to create structured evidence for matching, rather than to directly assign reviewers or make final peer-review decisions.

The novelty of this work lies in the design and evaluation of an auditable affinity-construction layer for reviewer matching. Unlike embedding-only systems that represent papers and reviewers primarily through dense vectors, our framework produces explicit keyword artifacts that remain human-readable and machine-parseable. Unlike graph-only approaches, the proposed pipeline combines graph-structured topical evidence with lexical and neural similarity signals under a unified evaluation protocol. Unlike recent LLM-assisted peer-review systems that use generative models for

broader reviewing or decision-support tasks, our approach deliberately restricts LLMs to controlled keyword induction through a heterogeneous producer ensemble and a stronger judge model. This design separates generative topic extraction from transparent scoring and downstream allocation, thereby improving inspectability while preserving compatibility with conventional reviewer-assignment infrastructure.

Affinity estimation is then performed using a family of transparent and complementary matchers. These include cosine similarity over TF-IDF or embedding representations; Jaccard similarity over normalized keyphrases; SBERT-based bi-encoder scoring; keyword-aware retrieval with re-ranking; and graph-based matchers over both global keyword co-occurrence graphs and document-local keyword graphs using Node2Vec embeddings with Jaccard-style fusion. These signals can be used directly for ranking or combined via late fusion before being passed to standard assignment solvers.

The proposed scores are designed to be *assignment-ready*. In a complete reviewer assignment system, each affinity score can be treated as the weight of a paper-reviewer edge, and a downstream optimizer can select binary assignment variables under operational constraints. Such constraints may require each paper to receive a fixed number of reviewers, impose lower and upper reviewer-capacity bounds, remove conflicted paper-reviewer pairs, and include fairness-aware objectives such as max-min paper coverage or workload-balancing penalties. Thus, the contribution of this paper is not to replace established constrained assignment solvers, but to improve and analyze the affinity matrix that those solvers optimize. Our experiments therefore focus on affinity quality, ranking behavior, and workload diagnostics, while the produced scores remain compatible with Hungarian-style matching, integer programming, and fairness-aware assignment formulations.

For evaluation, we follow established information retrieval practice and report MAP@3, MRR@3, and Precision@ k ($P@k$). Beyond ranking quality, we analyze conference-aware reviewer workload behavior by summarizing reviewer loads per conference and globally, and by quantifying workload inequality using the Gini coefficient. This allows us to assess whether high-scoring matching methods also produce realistic and balanced reviewer distributions, an important consideration before integrating affinity scores into a full operational assignment pipeline.

Specifically, this paper makes the following contributions.

- We introduce a modular, LLM-assisted reviewer-paper matching pipeline that restricts generative models to auditable *affinity construction*, while preserving transparency and compatibility with standard optimization-based assignment solvers.
- We propose an ensemble-based approach for keyword induction, in which multiple heterogeneous producer models generate candidate keyword sets and a stronger

judge model selects compact, machine-parseable representations, improving robustness and inspectability.

- We clarify and operationalize the separation between affinity construction and final constrained allocation: the proposed methods produce paper-reviewer scores that can be used as inputs to assignment solvers incorporating reviewer capacity, conflicts of interest, and fairness objectives.
- We provide a systematic empirical comparison of lexical, embedding-based, and graph-based affinity estimators—including keyword-aware variants—under a unified experimental framework and across multiple document representations.
- We introduce an evaluation protocol that jointly assesses retrieval effectiveness (MRR@3, MAP@3, and Precision@ k) and operational properties of reviewer assignment, including conference-aware workload realism and workload inequality via the Gini coefficient.
- We release a curated dataset of submissions and reviewer profiles to support reproducible benchmarking of reviewer matching methods.¹

Overall, this work demonstrates how LLM-assisted, *auditable* affinity construction can strengthen reviewer assignment by improving semantic matching while preserving transparency and compatibility with standard optimization-based pipelines. By combining LLM-guided keyword extraction and normalization with complementary lexical, embedding-based, and graph-based affinity estimators, our approach supports scalable reviewer assignment that can be evaluated not only for ranking quality but also for workload realism and fairness.

The remainder of this paper is organized as follows: Section II reviews related work. Section III defines the task addressed in this study. Section IV presents our LLM-assisted affinity-building and reviewer matching strategies, including keyword-aware, embedding-based, and graph-based methods. Section V describes the evaluation protocol and metrics, including both retrieval-oriented measures and workload/fairness criteria. Section VI reports and analyzes the experimental results. Finally, Section VII concludes the paper and outlines directions for future work.

II. RELATED WORK

Early work on the reviewer assignment problem (RAP) established a now-standard pipeline: represent submissions and reviewers, estimate paper-reviewer affinities, and solve a constrained optimization problem to allocate reviews. Deployed systems such as the Toronto Paper Matching System (TPMS) illustrate this approach at conference scale, combining language models, reviewer bids, and assignment optimizers [7], [23]. Subsequent systems validated these pipelines under practical constraints such as workload, conflicts of interest, and authority balancing [24], [25], [26]. These works provide the operational and optimization

foundations on which our study builds, but they generally treat affinity construction as a feature-engineering or supervised-scoring problem rather than as an auditable artifact-generation process.

Building on these foundations, later research focused on improving the representations used for affinity estimation. Early TF-IDF and topic-model approaches were gradually complemented by contextual encoders such as BERT, SciBERT, and citation-aware models such as SPECTER, which improved similarity estimation for scholarly text [8], [9], [27], [28]. More recent methods incorporate graph signals, contrastive training, and multi-factor objectives that jointly consider expertise, authority, diversity, and reviewer interest [29], [30]. These advances demonstrate the value of richer semantic representations; however, dense document embeddings are often difficult to audit because the reasons behind a high paper-reviewer score are not directly visible to program chairs or area chairs.

Knowledge-graph and network-aware approaches provide another important line of work. Such methods can exploit co-authorship, citation, venue, topic, or keyword co-occurrence structures to capture relationships that are not evident from surface text alone. They are especially attractive for reviewer assignment because they can encode relational evidence and support structural constraints. Nevertheless, graph-based similarity can be sensitive to graph construction choices, sparsity, and noisy expansion, and may produce concentrated reviewer workloads if high-degree or broadly connected reviewers dominate the ranking. This motivates our evaluation of both global and document-local graph variants alongside lexical and neural baselines, rather than treating graph structure as a universally superior signal.

Alongside methodological progress, surveys and benchmark efforts have clarified evaluation practices and open challenges in RAP, including fairness, scalability, and the lack of standardized datasets [31], [32], [33]. Established information retrieval evaluation principles further inform the interpretation of MAP@3, MRR@3, and Precision@ k in settings with incomplete or pooled judgments [34]. These insights motivate our joint evaluation of ranking quality and operational workload properties.

Fairness and robustness have become central themes in reviewer assignment. PeerReview4All provides provable fairness guarantees given similarity matrices, while related work studies local fairness, risk-averse objectives, and assignment under uncertainty [4], [35], [36]. Audits of peer-review pipelines further highlight the need for transparent and bias-aware similarity measures [37]. These perspectives inform our use of complementary similarity signals and workload-aware evaluation. In our setting, fairness-aware optimization is not replaced by the proposed affinity model; rather, the proposed scores are intended to provide a more interpretable affinity matrix that can subsequently be used by constrained assignment solvers.

Integrity and adversarial robustness are increasingly important as automated matchers are deployed. Recent

¹Dataset will be released upon acceptance.

studies show that targeted edits to abstracts or reviewer profiles can manipulate embedding-based matchers and evade bidding-based safeguards [38], [39], [40]. These findings motivate approaches that combine interpretable signals with diversified similarity measures rather than relying on a single embedding space. They also motivate the auditability requirement in our framework: extracted keywords can be inspected and compared against the source text, making suspicious or overly generic topical signals easier to detect than in purely dense-vector pipelines.

In parallel, LLMs have been explored as assistants in peer review and as evaluators, but empirical studies caution against autonomous or opaque uses due to bias, hallucination, and policy concerns [41], [42]. Recent LLM-assisted matching methods show that instruction-tuned models can generate useful intermediate representations or factor-specific signals for paper-reviewer matching [13]. Our work differs from such approaches by deliberately restricting LLMs to controlled keyword induction and by separating generative artifact construction from transparent scoring and downstream assignment. In other words, the LLM does not directly decide which reviewer should be assigned; it produces compact, inspectable evidence that can be used by classical, neural, and graph-based matchers.

Our keyword-centric approach is grounded in long-standing research on keyphrase extraction and generation. Classical methods such as RAKE, TextRank, and YAKE provide interpretable and computationally efficient baselines [43], [44], [45]. Neural and prompt-based approaches later reframed keyphrase generation as sequence modeling, ranking, and generate-then-select pipelines [46], [47], [48]. Recent evaluations show that modern LLM-based pipelines can outperform statistical baselines on scientific corpora while retaining some degree of interpretability through explicit phrase outputs [49]. These prior methods are directly relevant to the LLM component of our framework: traditional keyword extractors and single-model LLM configurations provide natural ablation points for assessing whether an ensemble-and-judge architecture offers benefits beyond simpler keyword-generation strategies.

At the same time, LLM-generated keyword profiles introduce their own limitations. Candidate keywords may hallucinate concepts not supported by the source text, omit important but less salient technical details, vary across model versions, or reflect biases inherited from the training data and prompting procedure. Reproducibility can also be affected by stochastic decoding, API changes, and undocumented model updates. For this reason, our pipeline constrains the LLM output format, stores intermediate candidate lists and judge decisions, applies deterministic post-processing, and evaluates the resulting profiles through downstream matching performance and workload diagnostics. These safeguards do not eliminate all risks, but they make the LLM contribution more auditable than direct generative reviewer assignment.

Taken together, prior work suggests that interpretable keyword signals can serve as a practical middle ground

between opaque embedding similarity and brittle flat topic lists. The gap addressed in this paper is therefore not simply the use of transformers, knowledge graphs, or LLMs for reviewer matching. Rather, we study an auditable affinity-construction framework in which a heterogeneous LLM producer ensemble and judge model generate compact keyword artifacts, and these artifacts are evaluated across lexical, neural, keyword-aware, and graph-based matchers under a unified protocol that also considers workload realism and inequality. This positioning distinguishes our approach from embedding-only systems, graph-only systems, and broader LLM-assisted peer-review tools, while preserving compatibility with established constrained assignment methods.

III. TARGETED TASK

The task addressed in this study is to estimate the topical affinity between a submitted paper and a pool of candidate reviewers, and to use these affinity scores to produce a ranked list of reviewers suitable for downstream assignment. We distinguish this affinity-estimation problem from the final constrained allocation problem. The present work focuses on constructing and evaluating paper-reviewer relevance scores; these scores can later be used by assignment solvers that impose reviewer-capacity limits, conflict-of-interest constraints, and fairness objectives. Let $\mathcal{P}$ denote the set of submitted papers and $\mathcal{R}$ the set of candidate reviewers. For each paper $p \in \mathcal{P}$, we consider three textual views: title, abstract summary, and full abstract. We denote a selected view by $v \in \mathcal{V} = \{\text{title, summary, abstract}\}$ and the corresponding text by $t_p^{(v)}$. Each reviewer $r \in \mathcal{R}$ is represented not by a single text, but by a set of prior publications $\mathcal{D}_r = \{d_{r,1}, \dots, d_{r,n_r}\}$, where each publication contributes textual evidence about the reviewer's expertise. This distinction leads to two different feature mappings: a paper mapping $f_p^{(v)}$ that maps one submission view into a paper profile, and a reviewer mapping f_r that aggregates evidence across multiple publications into an expertise profile.

A. FEATURE EXTRACTION AND SELECTION

Given a paper view $t_p^{(v)}$, the paper feature mapping is defined as:

$$f_p^{(v)}(p) = (K_p^{(v)}, x_p^{(v)}),$$

where $K_p^{(v)}$ is a normalized set of topical keywords and $x_p^{(v)}$ is an optional vector representation derived from the same text or keyword profile. The keyword set $K_p^{(v)}$ is generated through an LLM-based selection operator. Specifically, a set of producer models $\mathcal{M} = \{M_1, \dots, M_m\}$ receives the same input text and prompt specification and returns candidate keyword lists:

$$C_i(t_p^{(v)}) = \{k_{i,1}, \dots, k_{i,\ell_i}\}, \quad M_i \in \mathcal{M}.$$

A judge model J then selects the candidate list that best satisfies the criteria of topical coverage, specificity,

non-redundancy, and faithfulness to the source text:

$$i^* = J(t_p^{(v)}, C_1(t_p^{(v)}), \dots, C_m(t_p^{(v)})).$$

The selected list is then normalized through deterministic post-processing:

$$K_p^{(v)} = \text{Norm}(C_{i^*}(t_p^{(v)})),$$

where $\text{Norm}(\cdot)$ applies JSON parsing, lowercasing, whitespaces and punctuation trimming, lemmatization where appropriate, duplicate removal, and filtering of empty or malformed entries. This design connects the LLM component to feature engineering in a controlled way: the LLMs propose semantic descriptors, the judge selects the most faithful candidate set, and deterministic normalization converts the selected output into a stable representation for downstream matching. The reviewer feature mapping is different because reviewer expertise is inferred from a collection of prior publications rather than from a single submission. For each reviewer r , the LLM-based selection operator is applied to the textual evidence associated with each publication or publication summary $d_{r,j} \in \mathcal{D}_r$. The resulting reviewer keyword profile is obtained by aggregation:

$$K_r = \text{Agg}(\text{Norm}(C_{i^*}(d_{r,1})), \dots, \text{Norm}(C_{i^*}(d_{r,n_r}))).$$

where $\text{Agg}(\cdot)$ denotes union, deduplication, and optional frequency-aware consolidation across the reviewer's publications. The reviewer representation is then defined as:

$$f_r(r) = (K_r, x_r),$$

where x_r is an optional vector representation obtained from the aggregated reviewer profile. Thus, although the same controlled LLM selection mechanism is used to extract topical descriptors, the paper and reviewer mappings are not identical: $f_p^{(v)}$ summarizes the topical requirements of a single submitted paper view, whereas f_r aggregates multiple publication-level signals into a profile of reviewer expertise.

B. REVIEWER MATCHING

Reviewer matching is formulated as the computation of an affinity score between the paper representation and each reviewer representation:

$$s_m(p, r) = s_m(f_p^{(v)}(p), f_r(r)),$$

where s_m denotes a specific matching method. In this work, s_m may correspond to lexical overlap over keyword sets, TF-IDF cosine similarity, SBERT-based semantic similarity, keyword-aware neural retrieval, graph-based similarity over keyword co-occurrence structures, or a late-fusion score combining multiple signals. For each paper p , candidate reviewers are ranked in descending order of $s_m(p, r)$:

$$\pi_p = \text{argsort}_{r \in \mathcal{R}}(s_m(p, r)).$$

The resulting ranked list is the output of the affinity-estimation stage and can be inspected directly or passed to a downstream constrained assignment solver.

C. EVALUATION METRICS AND RELEVANCE LABELS

We evaluate matching quality as an information retrieval task. For each paper p , the relevant set R_p is defined as the set of reviewers historically assigned to that paper in the available conference data. We use these historical assignments as operational relevance labels because they provide observable paper-reviewer links produced by real peer-review workflows. However, these labels should not be interpreted as perfect ground-truth expertise judgments. In practice, reviewer assignments are affected by several factors beyond topical expertise, including reviewer availability, bidding behavior, workload balancing, conflicts of interest, reviewer seniority, and administrative decisions made by program chairs or area chairs. Therefore, our evaluation measures how well a method recovers historically observed assignments, not an absolute or complete notion of reviewer expertise. This limitation is important when interpreting the reported scores: a reviewer who was not historically assigned to a paper may still be qualified, and a historically assigned reviewer may have been selected partly for operational rather than purely topical reasons. Given the ranked list π_p and the historical relevant set R_p , we report standard ranking metrics, including Mean Reciprocal Rank at 3 (MRR@3), Precision at k (P@ k), and Mean Average Precision at 3 (MAP@3). These metrics assess whether historically assigned reviewers appear near the top of the recommendation list and how well the ranking aligns with the observed assignment data.

IV. REVIEWER ASSIGNMENT PIPELINE

Figure IV presents the complete data flow and operational sequence of the proposed reviewer-matching pipeline. The workflow contains two parallel input branches. The *paper branch* begins with submission records obtained from EasyChair and constructs a topical representation of each submitted paper from its title, abstract summary, or full abstract. The *reviewer branch* begins with publication records obtained from Semantic Scholar and constructs an expertise representation by aggregating evidence across the prior publications of each reviewer. The two branches use different feature mappings, as formally defined in Section III, but produce representations with a compatible output schema so that paper-reviewer affinity scores can be computed consistently.

The operational pipeline consists of the following stages:

- 1) **Data creation.** Submission records and reviewer-publication records are collected, anonymized, assigned stable identifiers, and serialized as linked JSON datasets.
- 2) **Canonical text construction.** Paper and reviewer texts are normalized using deterministic preprocessing. For each paper, title, abstract summary, and full abstract are retained as separate views. For each reviewer, the corresponding fields are retained at the publication level before aggregation.

- 3) **Role-specific keyword induction.** The paper mapping extracts keywords that characterize the topics and methodological expertise required to review a submission. The reviewer mapping separately extracts keywords that characterize the expertise demonstrated by each prior publication and then aggregates them into a reviewer profile. Although both mappings use the same ensemble-and-judge architecture, they use different role instructions and aggregation operations.
- 4) **Representation and affinity computation.** The resulting textual and keyword profiles are converted into lexical, neural, and graph-based representations. Each matching method produces a paper-reviewer affinity matrix whose entries quantify the relevance of reviewer r to paper p .
- 5) **Reviewer ranking.** For each paper, reviewers are sorted in descending order of affinity. Deterministic tie-breaking and predefined safeguards for empty keyword sets, zero vectors, and degenerate normalization ranges ensure reproducible outputs.
- 6) **Audit-trail generation.** Each stage emits inspectable artifacts, including normalized texts, producer outputs, judge selections, final keyword inventories, feature representations, score matrices, and top-3 rankings. These artifacts support debugging, controlled ablation, and reproducibility.

The output of this pipeline is an assignment-ready affinity matrix and a ranked reviewer list for each paper. The present study evaluates the quality and operational behavior of these affinities; final allocation under capacity, conflict-of-interest, and fairness constraints can subsequently be performed by a constrained assignment solver.

A. DATA CREATION

We construct two complementary datasets for reviewer assignment experiments: an *Authors' Dataset* of submitted papers and a *Reviewers' Dataset* describing the scholarly profiles of assigned reviewers. Source data are collected from EasyChair² (submissions and assigned reviewers) and Semantic Scholar³ (reviewer publications) following the collection procedure described in [50]. All records are serialized to JSON and linked through stable identifiers to support traceability across preprocessing, matching, and evaluation. Reviewer identities and candidate lists are anonymized in the released version using consistent pseudonymous IDs.

1) AUTHORS' DATASET

This dataset contains 663 papers and their assigned reviewers drawn from 85 computer science conferences organized or co-organized by the authors' institutions and collaborating organizations. The venues span Semantic Web and Linked Data, Artificial Intelligence and Machine Learning, Database and Information Systems, Computational Linguistics and

Natural Language Processing, and Ontologies/Knowledge Graphs, covering a broad spectrum of contemporary computer science. Each record is indexed by a unique key and includes: conference name, paper title, paper link, author list, abstract, and anonymized assigned reviewers.

2) REVIEWERS' DATASET

For each reviewer assigned in the Authors' Dataset, we queried the Semantic Scholar API⁴ to build a structured expertise profile. We retrieved up to 20 of the most-cited publications per reviewer and stored, for each publication: title, abstract, venue/link, total citations, and influential citations. Records are indexed by a stable reviewer identifier; internally, we maintain author IDs for disambiguation, while the released dataset exposes only anonymized identifiers. Each paper in the Authors' Dataset is assigned to three reviewers on average (663 papers, 1,989 assignments). After deduplication and profile resolution, the Reviewers' Dataset contains 524 unique reviewer profiles. This reduction reflects overlapping assignments, name disambiguation issues, incomplete Semantic Scholar profiles, and duplicate author pages.

3) ETHICS, ANONYMIZATION, AND AVAILABILITY

All personally identifying information is removed from the released data. Paper-reviewer links are preserved in anonymized form to enable reproducible evaluation. The anonymized JSON datasets and processing scripts are available upon reasonable request for non-commercial research use.

B. TEXT PREPROCESSING AND CANONICAL TEXT FIELDS

Preprocessing standardizes both corpora into analyzable units while preserving information critical for topical alignment. For papers, we combine the title, abstract, and a short abstract summary (generated by an LLM) into a structured record, retaining each field because titles often encode domain-specific head nouns (e.g., "contrastive learning"), while abstracts provide broader contextual and methodological signals; for reviewers, we aggregate short summaries of prior publications into a compact surrogate of expertise. We maintain a stable identifier for every paper and reviewer throughout, enabling deterministic serialization of intermediate artifacts as JSON dictionaries keyed by these IDs.

All texts are normalized with a deterministic cleaning pipeline. First, we lowercase characters and strip punctuation, retaining hyphenated morphology when it encodes meaningful compounds (e.g., "graph-based"). Second, we apply tokenization and stopword removal to suppress high-frequency function words that inflate lexical overlap without contributing to topical specificity. Third, we perform lemmatization (WordNet) to standardize inflectional variants ("models" → "model"), improving both set-based overlap (Jaccard) and vocabulary alignment for TF-IDF weighting.

²<https://www.easychair.org/>

³<https://www.semanticscholar.org/>

⁴<https://api.semanticscholar.org/api-docs/>

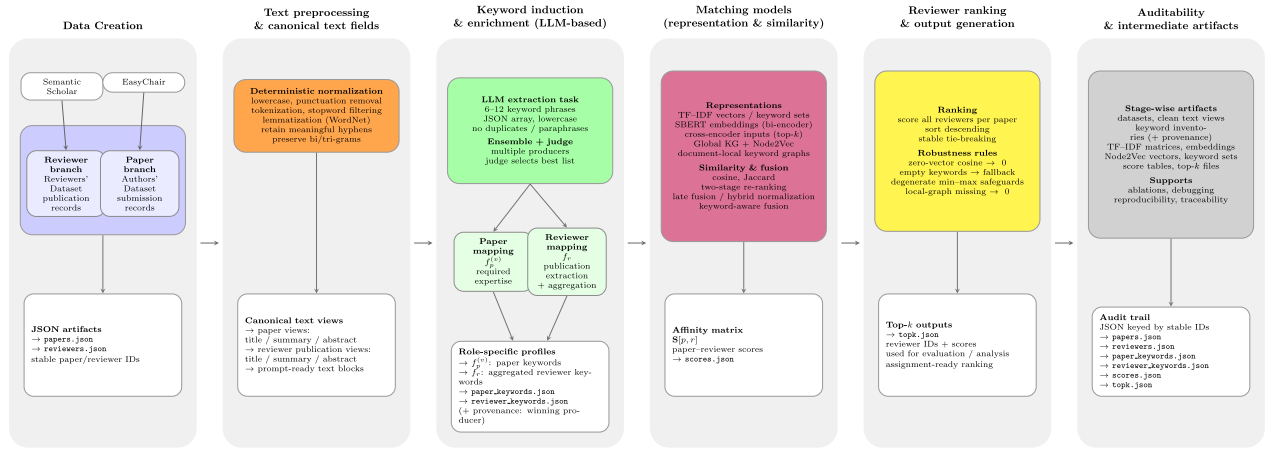


FIGURE 1. End-to-end reviewer assignment pipeline. The figure explicitly separates the paper branch, which starts from EasyChair submission records and produces $f_p^{(v)}$, from the reviewer branch, which starts from Semantic Scholar publication records and produces the aggregated reviewer profile f_r . The two branches converge in the matching stage, where the affinity matrix $S[p, r]$ is computed and then converted into deterministic top- k reviewer rankings. Each stage emits auditable artifacts keyed by stable paper and reviewer identifiers.

Because downstream topic extraction expects compact, multi-word expressions, we preserve bigrams and trigrams when they pass frequency thresholds (e.g., “knowledge graph”, “few-shot learning”); when such phrases are provided directly by the LLMs, we keep them intact. All transformations are applied uniformly to papers and reviewer profiles to avoid systematic biases that would otherwise distort similarity scores.

The output of preprocessing consists of two role-specific collections of canonical text views. For a paper p , preprocessing produces

$$T_p = \{t_p^{(\text{title})}, t_p^{(\text{summary})}, t_p^{(\text{abstract})}\},$$

where each view represents the topical content and methodological requirements of one submission. For a reviewer r with publication collection $\mathcal{D}_r = \{d_{r,1}, \dots, d_{r,n_r}\}$, preprocessing produces publication-level views

$$T_{r,j} = \{t_{r,j}^{(\text{title})}, t_{r,j}^{(\text{summary})}, t_{r,j}^{(\text{abstract})}\}, \quad d_{r,j} \in \mathcal{D}_r.$$

Reviewer texts are retained at publication level during keyword induction and are aggregated only after publication-level features have been extracted. This distinction implements the separate paper and reviewer mappings $f_p^{(v)}$ and f_r defined in Section III. Preprocessing emits normalized token streams, prompt-ready text fields, and stable paper, publication, and reviewer identifiers.

C. KEYWORD INDUCTION AND ENRICHMENT

Keyword induction converts the canonical text views into compact and auditable topical representations. The LLM component performs a narrowly defined feature-construction task; it does not compute paper-reviewer similarity, rank reviewers, or make assignment decisions. Its output is restricted to a machine-parseable list of topical phrases that is subsequently processed by deterministic components.

1) ROLE-SPECIFIC FEATURE CONSTRUCTION

Paper and reviewer profiles require different semantic instructions and different aggregation procedures. For a paper p and textual view v , producer model M_i receives a paper-specific prompt P_p and generates a candidate list

$$C_{p,i}^{(v)} = M_i(P_p(t_p^{(v)})).$$

The paper prompt asks for concepts that characterize the submission and the expertise needed to evaluate it.

For reviewer r , keyword extraction is performed separately for each prior publication $d_{r,j}$. Producer M_i receives a reviewer-specific prompt P_r and generates

$$C_{r,j,i}^{(v)} = M_i(P_r(t_{r,j}^{(v)})).$$

The reviewer prompt asks for concepts that characterize the expertise demonstrated by the publication. Publication-level keyword lists are subsequently consolidated:

$$K_r^{(v)} = \text{Agg}_{j=1}^{n_r} \text{Norm}(C_{r,j,i}^{(v)}),$$

where $\text{Norm}(\cdot)$ denotes deterministic normalization and $\text{Agg}(\cdot)$ performs union, deduplication, and frequency-aware consolidation across publications. In contrast, the paper profile is

$$K_p^{(v)} = \text{Norm}(C_{p,i}^{(v)}).$$

Thus, the two mappings share a compatible keyword schema but are not identical: $f_p^{(v)}$ represents the topical requirements of one submission, whereas f_r aggregates evidence of expertise across multiple publications.

2) PROMPT-DESIGN RATIONALE

The prompts were designed around four properties of LLM-based feature generation. First, because generative outputs can vary in length and format, the output is

bounded to 6–12 phrases and constrained to a strict JSON array. Second, because scientific matching depends strongly on multi-word technical concepts, the prompts request specific one-to-three-token phrases rather than unrestricted summaries. Third, because LLMs may produce generic or redundant expressions, the prompts explicitly prohibit generic terms, duplicates, and paraphrase variants. Fourth, because the paper and reviewer inputs play different semantic roles, purpose-specific prompts are used rather than applying one undifferentiated mapping to both corpora.

All producer models receive the same role-specific prompt template. We do not tune a separate prompt for each producer model or experimental matcher. Consequently, variation among producers reflects model behavior rather than model-specific prompt engineering, and downstream comparisons use the same selected keyword artifacts.

The paper prompt is:

“You are constructing a topical profile for reviewer matching. From the following paper text, return 6–12 compact and specific keyword phrases that describe the paper’s main research topics, methods, and the expertise required to review it. Use one-to-three-word phrases, avoid generic academic terms, and do not include duplicates or paraphrase variants. Return only a valid JSON array of lowercase strings. Text type: {title|summary|abstract}. Text: {text}.”

The reviewer-publication prompt is:

“You are constructing an expertise profile for reviewer matching. From the following publication text, return 6–12 compact and specific keyword phrases that describe the research expertise demonstrated by this publication. Use one-to-three-word phrases, avoid generic academic terms, and do not include duplicates or paraphrase variants. Return only a valid JSON array of lowercase strings. Text type: {title|summary|abstract}. Text: {text}.”

These prompts intentionally differ in their semantic target: the paper prompt represents required expertise, whereas the reviewer prompt represents demonstrated expertise.

3) PRODUCER ENSEMBLE AND JUDGE SELECTION

Let $\mathcal{M} = \{M_1, \dots, M_m\}$ denote the heterogeneous producer ensemble. Each producer independently receives the same source text and applicable role-specific prompt. The models therefore generate alternative candidate keyword profiles rather than repeated stochastic samples from a single model.

A separate judge model evaluates the indexed candidate lists against the source text. The judge is instructed to select the candidate that best satisfies four explicit criteria:

- 1) *faithfulness*: every selected concept must be supported by the source text;
- 2) *coverage*: the list should capture the main topics and methods;

- 3) *specificity*: technical concepts are preferred to generic descriptors; and
- 4) *non-redundancy*: duplicate and paraphrastic phrases should be avoided.

The judge instruction is:

“Given the source text and the indexed candidate keyword lists, select the single candidate that is most faithful, specific, complete, and non-redundant. Do not introduce new keywords and do not rewrite any candidate. Return only a JSON object containing the winning candidate index: {“winner”: integer}.”

For paper p , the selected producer is therefore:

$$i_p^* = J\left(i_p^{(v)}, C_{p,1}^{(v)}, \dots, C_{p,m}^{(v)}\right),$$

with an analogous publication-level selection $i_{r,j}^*$ for each reviewer publication. Restricting the judge to selection rather than free-form rewriting prevents the judge from introducing unsupported concepts after candidate generation.

The ensemble is implemented through parallel producer calls followed by one judge call. This structure is intended to expose complementary candidate profiles while limiting sequential latency. The producer prompt, judge prompt, model identifiers, raw candidate outputs, winning index, and final normalized list are retained as provenance artifacts.

4) DETERMINISTIC PARSING, NORMALIZATION, AND ENRICHMENT

The selected candidate is processed through a deterministic normalization pipeline. The pipeline validates the JSON structure, converts entries to strings, lowercases text, trims punctuation and whitespace, removes empty entries and duplicates, and applies controlled tokenization, stopword filtering, and lemmatization. Multi-word scientific expressions are preserved. Invalid or malformed entries are excluded rather than silently converted into new concepts.

Abbreviations are expanded only when the expansion is unambiguous and supported by the source text. Likewise, synonymic or parent-concept enrichment is permitted only when the corresponding concept is explicitly evidenced by the input. This source-conditioned restriction reduces semantic drift caused by aggressive expansion.

5) RELIABILITY, REPRODUCIBILITY, AND BIAS LIMITATIONS

LLM-generated keyword profiles remain subject to several limitations. A producer may hallucinate a concept that is absent from the source, omit a technically important but less salient contribution, or prefer terminology that reflects biases in its training data. Outputs may also change because of stochastic generation, provider-side model updates, or differences between model versions. In addition, the judge may systematically favor particular phrase styles or producer families.

The pipeline mitigates these risks through source-faithfulness instructions, bounded structured outputs,

role-specific prompts, ensemble generation, judge selection without keyword rewriting, deterministic normalization, and preservation of intermediate artifacts. Recording the prompt templates, provider-visible model identifiers, raw candidates, judge decision, and final keywords supports later auditing and rerunning. Nevertheless, these mechanisms reduce rather than eliminate hallucination, model drift, and bias; the extracted profiles should therefore be treated as inspectable affinity evidence rather than authoritative descriptions of paper content or reviewer competence.

6) EXTERNAL BASELINES AND PARTIAL LLM-COMPONENT ABLATION

To more precisely assess the contribution of the LLM-generated keyword representation, we compare the proposed ensemble-and-judge framework with several external and component-level baselines. Across all comparisons, the evaluation dataset, candidate-reviewer pool, ranking metrics, and top- k protocol are held constant to ensure comparability. The conventional lexical baselines are drawn from our earlier content-based reviewer-assignment study [50], which evaluated cosine similarity, dot-product similarity, and Jaccard similarity using paper titles, summaries, and abstracts. The GPT-4o-mini, BM25, KeyBERT, and SPECTER2 results are obtained from our previous LLM-based reviewer-matching study [51]. The evaluated baselines are summarized as follows:

- 1) **Previous lexical reviewer-assignment baselines:** cosine similarity, dot-product similarity, and Jaccard similarity applied to title-, summary-, and abstract-based representations;
- 2) **Single-LLM keyword baseline:** keyword generation using GPT-4o-mini, evaluated with cosine similarity, Jaccard similarity, and cosine-Jaccard average similarity;
- 3) **Non-generative keyword baseline:** keyword extraction using KeyBERT, evaluated under the same downstream similarity and ranking protocol;
- 4) **Lexical retrieval baseline:** BM25 ranking, included as a strong non-LLM information-retrieval reference; and
- 5) **Scientific-document embedding baseline:** SPECTER2-based matching, representing a domain-specific dense-representation approach for scientific documents.

These comparisons should be interpreted as an external-baseline and partial component ablation rather than as a complete producer-by-producer ablation. In particular, KeyBERT provides a non-generative keyword-extraction reference, BM25 provides a lexical retrieval reference, SPECTER2 provides a scientific-document embedding reference, and GPT-4o-mini provides a single-LLM keyword-generation reference. This allows us to quantify whether the proposed ensemble-and-judge framework provides benefits beyond simpler lexical, embedding-based, and single-model alternatives.

Because the stored artifacts from the ensemble stage retain the final judge-selected keyword list and winning producer identity but not all non-selected candidate lists for every producer, a complete retrospective single-producer ablation for all LLMs cannot be reconstructed. Therefore, we treat GPT-4o-mini as a single-LLM reference configuration and KeyBERT/BM25/SPECTER2 as external baselines, while interpreting the comparison as a partial ablation of the LLM keyword-induction component rather than as a complete producer-by-producer ablation. Quantitative results are reported in Section VI-E.

D. MATCHING MODELS (REPRESENTATION AND SIMILARITY)

All matching strategies compute an affinity score for each paper-reviewer pair by combining (i) a representation function (lexical vectors, dense embeddings, keyword sets, or graph embeddings) with (ii) a similarity function (cosine, Jaccard, two-stage cross-encoder re-ranking, and/or late fusion). Unless otherwise stated, keyword sets are the normalized, deduplicated 6–12 phrase inventories described in Section IV-C, and paper/reviewer texts are derived from the canonical fields described in Section IV-B. In what follows, we describe each matching model by specifying its inputs, representation construction, scoring function, and any fusion or normalization used to produce the final ranking.

1) LEXICAL KEYWORD MATCHING

This model provides a strong lexical baseline by scoring pairs using both weighted bag-of-keywords overlap and exact set overlap derived from induced keyword inventories.

Given keyword inventories induced for each paper and reviewer, we represent each entity in two complementary lexical forms: (i) a non-negative TF-IDF vector in a shared vocabulary, and (ii) a discrete keyword set for exact overlap. TF-IDF vectors are built using character and word n -grams on the cleaned keyword lists, and (when operating on text fields directly) we use the clean token stream produced by preprocessing. Let V denote the shared vocabulary fit on the combined paper-reviewer corpus. Each paper d and reviewer r receive TF-IDF vectors $x, y \in \mathbb{R}_{\geq 0}^{|V|}$, and lemmatized, deduplicated keyword sets $P, R \subseteq V$ (where the set view is derived from the canonical keyword strings after normalization).

Similarity is computed via cosine similarity in TF-IDF space and Jaccard similarity over keyword sets, optionally fused by a simple arithmetic mean. Cosine similarity is

$$S_{\cos}(x, y) = \frac{x \cdot y}{\|x\|_2 \|y\|_2} = \frac{\sum_{t \in V} x_t y_t}{\sqrt{\sum_{t \in V} x_t^2} \sqrt{\sum_{t \in V} y_t^2}}.$$

With TF-IDF features,

$$x_t = \text{tf}(t, d) \cdot \log\left(\frac{N}{1 + \text{df}(t)}\right),$$

and analogously for y , where N is the number of documents used to fit the vectorizer and $\text{df}(t)$ counts documents

containing term t . Because TF-IDF weights are non-negative, $S_{\cos}(x, y) \in [0, 1]$. We apply L_2 normalization (implicitly via the denominator), and define $S_{\cos}(x, y) = 0$ if either vector is the zero vector (e.g., all keywords filtered). Jaccard similarity is

$$S_{\text{Jac}}(P, R) = \frac{|P \cap R|}{|P \cup R|} \in [0, 1],$$

where P and R are sets of normalized n -gram keywords (1–3 tokens) after lowercasing, lemmatization, and punctuation stripping. Finally, we compute a late-fusion combined similarity:

$$\text{Combined Similarity} = \frac{S_{\cos}(x, y) + S_{\text{Jac}}(P, R)}{2}.$$

This late-fusion scheme is scale-compatible and interpretable: reviewers must be simultaneously close in TF-IDF space and share explicit keywords to score highly.

2) NEURAL TEXT MATCHING (BI-ENCODER)

This model captures semantic relatedness between papers and reviewers using dense sentence embeddings and cosine similarity, enabling efficient all-pairs scoring.

We embed paper and reviewer texts using a sentence-transformer (all-MiniLM-L6-v2).⁵ For a paper p , we encode a single text representation (title, abstract, or abstract summary, depending on the experimental condition) to obtain $\mathbf{e}_p$. For a reviewer r , we encode each of the reviewer's publication texts and represent the reviewer by mean-pooling publication embeddings followed by L_2 normalization to obtain $\mathbf{e}_r$. The affinity score is computed as cosine similarity between the L_2 -normalized embeddings,

$$s_{\text{text}}(p, r) = \cos(\mathbf{e}_p, \mathbf{e}_r), \quad (1)$$

which is evaluated efficiently as a dot product between normalized vectors.

3) NEURAL RE-RANKING (TWO-STAGE)

This model improves ranking quality by first retrieving a small candidate set with a bi-encoder and then re-ranking those candidates with a cross-encoder that jointly models each paper-reviewer pair.

a: CANDIDATE GENERATION (STAGE 1)

We compute the bi-encoder score $s_{\text{text}}(p, r)$ for all reviewers and retain the top-3 candidates per paper. To ensure that keywords contribute explicitly to matching, we augment inputs for both papers and reviewers: papers include an appended line of LLM-extracted keywords, while reviewers are represented by a truncated profile that concatenates snippets from a limited number of publications together with an aggregated keyword list, mitigating excessive input length for cross-encoder inference.

b: CROSS-ENCODER SCORING (STAGE 2)

We re-rank only the top-3 candidates using a cross-encoder that jointly models the paper-reviewer pair (cross-encoder/ms-marco-MiniLM-L-6-v2),⁶ producing a score $s_{\text{ce}}(p, r)$.

c: SCORE NORMALIZATION AND HYBRID FUSION

Since s_{text} and s_{ce} are not directly comparable in scale, we apply per-paper min-max normalization,

$$\text{norm}(x) = \frac{x - \min(x)}{\max(x) - \min(x)}, \quad (2)$$

with standard safeguards against degenerate ranges. We then compute a hybrid score via a convex combination,

$$s_{\text{hybrid}}(p, r) = \lambda \text{norm}(s_{\text{ce}}(p, r)) + (1 - \lambda) \text{norm}(s_{\text{text}}(p, r)), \quad (3)$$

where λ controls the influence of re-ranking. Reviewers outside the top-3 candidate set retain their normalized bi-encoder score, enabling a complete ranking while limiting cross-encoder evaluations.

4) KEYWORD-AWARE NEURAL VARIANTS

These variants retain the SBERT backbone while incorporating LLM-induced keyword inventories as explicit signals, either by embedding keyword lists directly or by adding a soft coverage term that encourages topical coverage of a paper's keywords by a reviewer's expertise.

a: KEYWORD-ONLY EMBEDDINGS (DUAL-SBERT SIMILARITY)

We compute the text-based similarity $s_{\text{text}}(p, r)$ as above. Independently, we compute keyword-only embeddings by encoding each keyword string with SBERT and mean-pooling within each entity. For papers, we average embeddings of the paper's keyword list to obtain $\mathbf{e}_p^{\text{kw}}$; for reviewers, we average embeddings of the union of keywords across their publications to obtain $\mathbf{e}_r^{\text{kw}}$. The resulting keyword similarity is

$$s_{\text{kw}}(p, r) = \cos(\mathbf{e}_p^{\text{kw}}, \mathbf{e}_r^{\text{kw}}), \quad (4)$$

with a safe fallback embedding when an entity has an empty keyword set. Because s_{text} and s_{kw} may differ in empirical range, we optionally apply per-paper min-max normalization across reviewers,

$$\tilde{s}(p, r) = \frac{s(p, r) - \min_r s(p, r)}{\max_r s(p, r) - \min_r s(p, r)}, \quad (5)$$

again with guards against zero denominators. Under this normalization, we define the combined score as

$$s_{\text{avg}}(p, r) = \frac{1}{2} (\tilde{s}_{\text{text}}(p, r) + \tilde{s}_{\text{kw}}(p, r)), \quad (6)$$

and otherwise average the raw cosine similarities.

⁵<https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>

⁶<https://huggingface.co/cross-encoder/ms-marco-MiniLM-L-6-v2>

b: IDF-WEIGHTED SOFT KEYWORD COVERAGE (SBERT WITH SOFT COVERAGE FUSION)

We augment SBERT semantic matching with an asymmetric, soft keyword-coverage score that measures how well a reviewer's expertise covers a paper's keyword set. We first compute inverse document frequency (IDF) over reviewers:

$$\text{idf}(k) = \log\left(\frac{N+1}{\text{df}(k)+1}\right) + 1, \quad (7)$$

where N is the number of reviewers and $\text{df}(k)$ is the number of reviewers whose keyword set contains k . Next, we embed the union of all paper and reviewer keywords once (using the same SBERT encoder), obtaining keyword vectors $\mathbf{v}_k$. For a paper p and reviewer r , we align each paper keyword $k \in K_p$ to its best-matching reviewer keyword $k' \in K_r$ and compute an IDF-weighted coverage score:

$$s_{\text{softkw}}(p, r) = \frac{\sum_{k \in K_p} \text{idf}(k) \cdot \max_{k' \in K_r} \max(0, \cos(\mathbf{v}_k, \mathbf{v}_{k'}))}{\sum_{k \in K_p} \text{idf}(k)}. \quad (8)$$

In practice, keyword sets are small (typically 6–12 items), making this alignment computation negligible relative to SBERT encoding. We clamp negative cosine values to 0 to preserve a coverage interpretation. Finally, we fuse semantic similarity and soft coverage using a paper-dependent weight:

$$s_{\text{fusion}}(p, r) = \alpha(p) s_{\text{text}}(p, r) + (1 - \alpha(p)) s_{\text{softkw}}(p, r), \quad (9)$$

where $\alpha(p) \in [0.3, 0.7]$ increases with $|K_p|$, reflecting greater reliance on semantic text similarity when more keywords are available.

5) GLOBAL KG-BASED MATCHING

This model leverages corpus-level keyword co-occurrence structure by constructing a single global keyword graph and scoring paper-reviewer pairs using both embedding-based and graph-overlap measures.

a: REPRESENTATION: GLOBAL KEYWORD CO-OCCURRENCE GRAPH AND NODE2VEC EMBEDDINGS

We build a single undirected weighted KG ($G = (V, E, w)$) over the filtered keywords, where each keyword corresponds to a node. An undirected edge is added between two keywords if they co-occur within the same instance (within a paper record or within the aggregated keyword bag of a reviewer across publications). Edge weights count co-occurrence frequency:

$$w(u, v) \leftarrow w(u, v) + 1. \quad (10)$$

We train Node2Vec on the global KG to obtain an embedding $\mathbf{z}_k \in \mathbb{R}^d$ for each keyword node k . A paper or reviewer is embedded by mean-pooling the vectors of its in-vocabulary keywords:

$$\mathbf{e}(D) = \frac{1}{|K_D|} \sum_{k \in K_D} \mathbf{z}_k, \quad (11)$$

where reviewer keyword sets are formed by aggregating keywords across all of a reviewer's publications prior to pooling. This global construction yields a single embedding space shared by all entities.

b: SIMILARITY: COSINE, EXPANDED-NODE JACCARD, AND EDGE-BASED METRICS

We compute three KG-based similarities.

- *Node2Vec cosine similarity*: for each paper p and reviewer r with valid embeddings,

$$s_{\text{cos}}(p, r) = \cos(\mathbf{e}_p, \mathbf{e}_r). \quad (12)$$

For efficiency, reviewer embeddings are stacked into a normalized matrix so that all reviewer scores for a paper can be computed via a matrix-vector product.

- *KG-expanded node Jaccard*: to capture related topics not present verbatim in the original keyword sets, we expand each keyword set by traversing the KG to a shallow depth (one hop), while considering a bounded number of neighbors per node to limit noise [52], [53]. Let $\hat{K}_p$ and $\hat{K}_r$ denote the expanded sets (including originals); we compute

$$s_{\text{nodeJ}}(p, r) = \frac{|\hat{K}_p \cap \hat{K}_r|}{|\hat{K}_p \cup \hat{K}_r|}. \quad (13)$$

Reviewer expansions are precomputed once, while paper expansions are computed per paper during scoring.

- *KG edge Jaccard*: we measure structural similarity between keyword-induced subgraphs via an edge-set Jaccard score. For a node set K , let $E(K)$ denote the set of undirected edges in the global KG whose endpoints are both contained in K . Using expanded sets,

$$s_{\text{edgeJ}}(p, r) = \frac{|E(\hat{K}_p) \cap E(\hat{K}_r)|}{|E(\hat{K}_p) \cup E(\hat{K}_r)|}. \quad (14)$$

Reviewer edge sets are precomputed once, whereas paper edge sets are constructed on-the-fly during scoring.

c: FUSION

In addition to reporting each metric independently, we consider a combined score that averages cosine and expanded node-Jaccard:

$$s_{\text{comb}}(p, r) = \frac{1}{2} (s_{\text{cos}}(p, r) + s_{\text{nodeJ}}(p, r)). \quad (15)$$

6) DOCUMENT-LOCAL KEYWORD GRAPHS

This variant represents each paper and reviewer with a document-specific keyword co-occurrence graph and applies Node2Vec per instance, serving primarily as a lightweight sanity-check baseline.

a: REPRESENTATION: PER-DOCUMENT GRAPHS AND NODE2VEC EMBEDDINGS

In this variant, each paper and each reviewer is represented by its own *document-local* keyword co-occurrence graph. Keywords are extracted from the `keywords` field, normalized by string casting (if needed), trimming whitespace, dropping empty entries, and deduplication. For reviewers, keywords are obtained by aggregating `keywords` across all of the reviewer's publications into a single deduplicated set, and the same cleaning procedure is applied.

Given a cleaned keyword set $K = \{k_1, \dots, k_m\}$ for a paper or reviewer, we build an undirected graph $G = (V, E)$ where $V = K$ and every pair of distinct keywords is connected (clique co-occurrence):

$$E = \{(k_i, k_j) \mid i < j\}. \quad (16)$$

Instances with fewer than two keywords are skipped because no meaningful co-occurrence structure can be formed. For each document-local graph, we train a Node2Vec model (embedding dimension $d = 64$, walk length 10, 50 walks per node, skip-gram window size 5) to learn node embeddings $\mathbf{z}_v \in \mathbb{R}^d$ for $v \in V$. We then compute a single graph-level embedding via mean pooling:

$$\mathbf{e}(G) = \frac{1}{|V|} \sum_{v \in V} \mathbf{z}_v. \quad (17)$$

This yields one embedding per retained paper and one embedding per retained reviewer.

b: CAVEAT: EMBEDDING SPACES DIFFER

Because embeddings are learned independently per graph, the cosine similarity below should be interpreted as a lightweight heuristic baseline rather than a globally aligned representation model. Accordingly, this variant is intended as a sanity-check baseline rather than a competitive representation model, with the Jaccard component providing the primary semantically grounded signal.

c: SIMILARITY AND FUSION: COSINE, JACCARD, AND LATE FUSION

We compute (i) cosine similarity between pooled graph-level embeddings,

$$s_{\cos}(p, r) = \cos(\mathbf{e}(G_p), \mathbf{e}(G_r)), \quad (18)$$

and (ii) Jaccard similarity between raw keyword sets,

$$s_J(p, r) = \frac{|K_p \cap K_r|}{|K_p \cup K_r|}. \quad (19)$$

Entities skipped due to insufficient keywords are excluded from the graph-based similarity computation and assigned zero similarity in fused scores. Finally, we compute an unweighted fused score:

$$s_{\text{comb}}(p, r) = \frac{1}{2} (s_{\cos}(p, r) + s_J(p, r)), \quad (20)$$

and rank reviewers in descending order of each metric.

7) COMPUTATIONAL COMPLEXITY, SCALABILITY, AND DEPLOYMENT CONSIDERATIONS

Reviewer-assignment systems must be evaluated not only by ranking quality but also by their computational cost and scalability. Let $P = |\mathcal{P}|$ denote the number of submitted papers, $R = |\mathcal{R}|$ the number of candidate reviewers, d the embedding dimension, and K the maximum number of keywords retained for a paper or reviewer profile. In our setting, K is small because keyword inventories are restricted to compact 6–12 phrase lists. The computational cost of the proposed pipeline can be separated into offline representation construction and online paper–reviewer scoring.

a: LLM-BASED KEYWORD INDUCTION

The LLM-based keyword-induction stage is the most expensive upstream component, but it is performed offline and its outputs are cached. If N_{in} denotes the number of processed paper views and reviewer-publication views, and M denotes the number of producer models, the ensemble-and-judge architecture requires M producer calls and one judge call per input text. The number of LLM calls therefore scales as

$$O(N_{\text{in}}(M + 1)).$$

Parallel producer execution reduces wall-clock latency but does not reduce total token usage or monetary cost. Once keyword inventories are generated, the normalized keyword lists, winning producer identities, judge decisions, and prompt metadata are stored and reused by all downstream matchers. In a deployed system, reviewer profiles can be cached and updated periodically, so only new submissions and newly added or modified reviewer publications require fresh LLM processing.

b: LEXICAL AND KEYWORD-SET METHODS

TF-IDF and BM25-style lexical methods are among the most scalable alternatives. After sparse vector construction, scoring is efficient because it depends on the number of non-zero terms rather than the full vocabulary size. Jaccard similarity over keyword sets has expected cost $O(K)$ per paper–reviewer pair when hashed sets are used, yielding $O(PRK)$ total pairwise scoring cost. Since K is bounded and small, exact keyword-overlap scoring remains inexpensive for conference-scale reviewer pools.

c: DENSE EMBEDDING AND NEURAL RETRIEVAL METHODS

For SBERT and SPECTER2-style bi-encoder methods, paper and reviewer texts are encoded once into dense vectors. After L_2 normalization, all paper–reviewer cosine similarities can be computed using matrix multiplication with cost $O(PRd)$. The embeddings can be cached, and large reviewer pools can be handled using blocked matrix multiplication or approximate nearest-neighbor indexing when storing the full $P \times R$ affinity matrix is unnecessary.

Two-stage neural re-ranking has a different cost profile. The bi-encoder first retrieves a shortlist of K_0 candidate

reviewers for each paper, and the cross-encoder is applied only to those candidates. Its inference cost is therefore proportional to $O(PK_0C_{ce})$, where C_{ce} is the cost of one cross-encoder forward pass. This is substantially cheaper than applying the cross-encoder to all PR paper-reviewer pairs, but it is still more expensive than pure lexical or bi-encoder scoring.

d: KEYWORD-AWARE NEURAL METHODS

Keyword-only SBERT similarity requires encoding the unique keyword strings once and then mean-pooling them into paper and reviewer keyword embeddings. Pairwise scoring again scales as $O(Prd)$. The IDF-weighted soft keyword-coverage score aligns each paper keyword with the closest reviewer keyword. Its direct cost per pair is $O(|K_p||K_r|d)$, and the worst-case total cost is $O(PRK^2d)$. Because the keyword lists are intentionally short, this cost remains manageable, and reviewer keyword vectors and IDF values can be precomputed.

e: KNOWLEDGE-GRAPH-BASED METHODS

For the global keyword graph, graph construction requires adding co-occurrence edges among keywords appearing in the same entity profile. If K_D is the keyword set of entity D , graph construction scales as

$$o\left(\sum_D |K_D|^2\right).$$

Because keyword sets are bounded, this stage grows approximately linearly with the number of entities. Node2Vec training is performed once on the global graph, after which paper and reviewer graph embeddings are obtained by mean pooling. Pairwise cosine scoring over the resulting embeddings costs $O(Prd)$. Expanded-node and edge-Jaccard similarities require additional set operations, but reviewer expansions and reviewer edge sets can be precomputed and reused across papers.

The document-local graph variant is less scalable. It constructs and embeds a separate graph for every paper and reviewer, so Node2Vec training is repeated per entity. Moreover, because the embedding spaces are learned independently, the cosine component is only a heuristic baseline rather than a globally aligned semantic representation. For this reason, document-local graph matching is useful mainly as a diagnostic comparison and is not the preferred option for latency-sensitive deployment.

f: PRACTICAL DEPLOYMENT CONSIDERATIONS

A practical deployment should separate offline and online computation. Reviewer-publication collection, LLM keyword induction, reviewer-profile aggregation, embedding generation, and global graph construction can be performed offline and cached. At submission time, the system needs to process only the new paper text, compute its representation, and score it against cached reviewer profiles. Lexical, BM25,

Jaccard, and bi-encoder methods provide the lowest online cost. Cross-encoder re-ranking should be restricted to a small candidate shortlist, and document-local Node2Vec should be avoided for real-time use unless latency is not a concern.

Finally, computational cost should be considered together with auditability. Although LLM keyword induction is more expensive than lexical or embedding-only baselines, it produces compact intermediate artifacts that can be inspected, cached, reused, and compared across matchers. This makes the additional inference cost acceptable when transparency and traceability are central deployment requirements.

E. REVIEWER RANKING AND OUTPUT GENERATION

Given any matcher score definition $s(p, r)$ from Section IV-D, including cosine similarity, Jaccard similarity, late fusion, SBERT cosine similarity, cross-encoder hybrid scoring, and KG-based similarities, the pipeline computes affinity scores between each paper and all candidate reviewers, sorts reviewers by score in descending order, and returns the top-3 candidates for each paper. For TF-IDF cosine similarity, computation is linear in the number of non-zero TF-IDF entries and is implemented using sparse operations; for Jaccard similarity, computation is linear in $|P| + |R|$ through hashed set intersections; for SBERT cosine similarity, scores are computed as matrix products over L_2 -normalized embeddings; and for two-stage re-ranking, cross-encoder inference is restricted to the Stage 1 top-3 candidate set.

1) POSITION IN THE END-TO-END DATA FLOW

This stage receives the paper-reviewer affinity scores produced by the matching models and converts them into deterministic, serializable rankings. Let $\mathcal{P}$ denote the set of papers and $\mathcal{R}$ the candidate-reviewer pool. The complete output of a matcher can be represented as an affinity matrix

$$\mathbf{S} = [s(p, r)]_{p \in \mathcal{P}, r \in \mathcal{R}}. \quad (21)$$

Each row of $\mathbf{S}$ contains the scores of all reviewers for one paper. Ranking is therefore performed row by row, after which the highest-scoring k reviewers are retained for retrieval evaluation and inspection. This establishes an explicit connection between the representation and similarity stage, the ranking stage, and the downstream assignment stage illustrated in Figure IV.

2) TIE-BREAKING AND DETERMINISTIC ORDERING

Rankings are determined by descending score; ties are resolved deterministically using stable reviewer identifiers, represented by consistent pseudonymous IDs in the released dataset, to ensure reproducibility across runs and facilitate auditing.

The tie-breaking policy is applied uniformly to all evaluated matchers. Consequently, two methods that produce identical affinity values are not allowed to differ merely because of non-deterministic sorting behavior. The selected matcher, score value, reviewer identifier, and final rank position are preserved in the ranking artifact.

3) HANDLING MISSING DATA AND DEGENERATE REPRESENTATIONS

When TF-IDF vectors are zero, for example because all keywords have been filtered, cosine similarity is defined as 0. In keyword-aware neural variants, keyword-only embeddings use a safe fallback representation when an entity has an empty keyword set. In document-local keyword graphs, instances with fewer than two keywords are skipped during graph construction; such entities are excluded from graph-based cosine computation and assigned zero similarity in fused scores. For min-max normalization in cross-encoder hybrid fusion and optional dual-SBERT fusion, standard safeguards are applied when $\max(x) = \min(x)$.

The pipeline records these exceptional cases instead of silently discarding them. In particular, the score artifact can indicate whether a value resulted from a valid representation, an empty keyword profile, a zero vector, a missing local graph, or a degenerate normalization range. This information supports error analysis and prevents missing-data behavior from being confused with genuine evidence of low reviewer relevance.

4) FINAL OUTPUT FORMAT

For each paper, the ranking stage produces a top-3 list of reviewer identifiers, rank positions, and affinity scores under the selected matcher. When required for comparative analysis, the exported record also contains the component scores used by a fused method, such as text similarity, keyword similarity, graph similarity, or cross-encoder score. A representative JSON structure is

```
{
  "paper_id": [
    {
      "reviewer_id": "reviewer_001",
      "rank": 1,
      "score": 0.8421,
      "matcher": "keyword_aware_sbert"
    }
  ]
}
```

The complete score matrix is retained separately from the truncated top-3 output. The top-3 file is used for ranking-based evaluation, whereas the complete matrix can be supplied to a constrained reviewer-assignment optimizer. This distinction is important because the best reviewer for an individual paper cannot always be assigned when reviewer capacities, conflicts of interest, and global fairness requirements are considered.

5) INTEGRATION WITH CONSTRAINED REVIEWER ASSIGNMENT

The ranked list is an intermediate recommendation artifact rather than the final allocation. In an operational system, the complete affinity matrix $\mathbf{S}$ is combined with venue-specific

constraints. Let $x_{pr} \in \{0, 1\}$ indicate whether reviewer r is assigned to paper p . A downstream solver may maximize total affinity

$$\max_{\mathbf{x}} \sum_{p \in \mathcal{P}} \sum_{r \in \mathcal{R}} s(p, r) x_{pr}, \quad (22)$$

subject to requirements such as assigning q_p reviewers to each paper, respecting reviewer-capacity limits, and excluding known conflicts of interest:

$$\sum_{r \in \mathcal{R}} x_{pr} = q_p, \quad \forall p \in \mathcal{P}, \quad (23)$$

$$\ell_r \leq \sum_{p \in \mathcal{P}} x_{pr} \leq u_r, \quad \forall r \in \mathcal{R}, \quad (24)$$

$$x_{pr} = 0 \quad \text{for every conflicted pair } (p, r). \quad (25)$$

Fairness-aware objectives can additionally discourage concentrated workloads or maximize the minimum affinity received by any paper. Thus, the proposed matchers provide the affinity evidence optimized by a complete assignment framework, while capacity, conflict, and fairness policies remain explicit and configurable downstream constraints.

The experiments in this paper intentionally focus on affinity construction and ranking quality so that the effects of lexical, neural, keyword-aware, and graph-based representations can be evaluated independently of venue-specific allocation policies. We therefore do not claim to provide a comparative benchmark of assignment solvers. Instead, the workload and Gini analyses serve as assignment-side diagnostics, and the exported affinity matrices provide the required interface for future end-to-end evaluation under capacity, conflict-of-interest, and fairness constraints. Because complete conflict-of-interest and reviewer-capacity metadata were unavailable in the anonymized dataset, this study reports the constrained-assignment interface and workload diagnostics rather than a full solver benchmark.

F. AUDITABILITY AND INTERMEDIATE ARTIFACTS

Auditability is a central design requirement of the proposed pipeline. As illustrated in Figure IV, each operational stage consumes an explicit input artifact and emits a serializable output artifact keyed by stable paper, publication, and reviewer identifiers. The resulting data flow is

$$\begin{aligned} \text{raw data} &\rightarrow \text{canonical text} \rightarrow \text{keyword profiles} \\ &\rightarrow \text{representations} \rightarrow \text{affinity scores} \rightarrow \text{rankings} \end{aligned} \quad (26)$$

Data collection produces dataset-level files such as `papers.json` and `reviewers.json`. Preprocessing emits normalized token streams and prompt-ready title, summary, and abstract fields. Keyword induction stores paper-level and publication-level keyword inventories, together with the producer candidates, selected producer identity, judge decision, and normalized final list. Feature construction exports TF-IDF matrices, SBERT embeddings, Node2Vec vectors, keyword sets, and graph representations.

Similarity computation exports per-matcher paper–reviewer score tables, while the ranking stage emits deterministic top-3 recommendation files.

1) PROMPT AND MODEL PROVENANCE

For LLM-assisted keyword induction, reproducibility requires more than storing the final keywords. The audit record therefore preserves the applicable paper or reviewer prompt template, the input text type, provider-visible producer and judge model identifiers, raw candidate lists, the winning candidate index, the selected producer, and the output after deterministic normalization. This provenance makes it possible to determine whether a downstream ranking change originated from the source data, prompt instructions, producer candidates, judge selection, normalization procedure, or matching function.

2) SUPPORT FOR CONTROLLED ABLATION

The explicit artifact boundaries permit one component to be replaced without recomputing unrelated stages. For example, traditional keyword extraction, a single LLM producer, and the complete ensemble-and-judge configuration can be compared using the same preprocessed texts, reviewer pool, similarity function, ranking policy, and relevance labels. Similarly, different matchers can consume one fixed set of keyword profiles. This separation is necessary to attribute performance differences to the intended component rather than to simultaneous changes in keyword generation and similarity computation.

3) RISKS OF LLM-GENERATED KEYWORD PROFILES

The presence of an audit trail does not eliminate the limitations of LLM-generated profiles. A producer may introduce a keyword unsupported by the source text, omit a technically important concept, or select terminology influenced by biases in its training data. The judge may also prefer a particular producer family, phrase style, or level of specificity. In addition, repeating the same pipeline at a later date may yield different candidates because of stochastic generation, provider-side model updates, or changes in model versions. Such changes can propagate into similarity scores and reviewer rankings.

The framework mitigates these risks through role-specific prompts, strict machine-parseable outputs, source-faithfulness criteria, judge selection without free-form rewriting, deterministic normalization, and preservation of all intermediate candidates and decisions. A suspicious keyword can therefore be traced back to the responsible input, producer output, and judge choice and compared directly with the source text. Nevertheless, these safeguards reduce rather than eliminate hallucination, model drift, reproducibility limitations, and bias. The keyword profiles should consequently be treated as inspectable affinity evidence rather than authoritative descriptions of paper content or reviewer competence.

4) HUMAN OVERSIGHT AND DEPLOYMENT USE

In practical deployment, the audit artifacts allow program chairs to inspect why a reviewer received a high score, identify unsupported or overly generic keywords, and override inappropriate recommendations before final allocation. They also support periodic re-evaluation when a provider changes a model version or when reviewer profiles are updated. Final assignments should therefore remain subject to human oversight and explicit venue policies, particularly for conflicts of interest, reviewer availability, confidentiality, capacity, and fairness.

Overall, the stage-wise artifact design supports systematic debugging, controlled ablation, provenance tracking, verification, and re-analysis. It also makes clear where the LLM component ends and where deterministic similarity computation, ranking, and constrained allocation begin.

V. EVALUATION

This section describes how we assess reviewer-recommendation quality and summarizes the experimental protocol used throughout the study. We treat each submitted paper as a query and use its historically assigned reviewers as an *observed relevance set*. We deliberately avoid interpreting these assignments as definitive observed relevance labels because real conference assignments reflect both expertise and operational considerations.

Given a paper d , the system returns a ranked list of candidate reviewers

$$\pi_d = (\pi_d(1), \pi_d(2), \dots).$$

We compute metrics independently for each paper and then macro-average them across all evaluable papers, so that each submission contributes equally to the final score. Unless stated otherwise, we report top-3 variants with $K = 3$, emphasizing the portion of the ranking most likely to be inspected by area chairs and program chairs.

Let R_d^{hist} denote the set of reviewers historically assigned to paper d . These assignments constitute the only directly observable paper–reviewer links in our dataset and therefore provide a practical supervision signal for retrospective evaluation. However, they are influenced by factors beyond topical expertise, including reviewer availability, bidding behavior, workload balancing, conflicts of interest, reviewer seniority, geographic or institutional considerations, and administrative decisions made by program chairs. Consequently, our evaluation measures agreement with historically observed assignment decisions rather than an absolute or exhaustive notion of reviewer suitability. A reviewer absent from R_d^{hist} may still be qualified, and a historically assigned reviewer may have been selected partly because of operational constraints.

The reported metrics should therefore be interpreted as measures of historical-assignment recovery. This limitation is particularly important because the available labels are incomplete: only the reviewers who were ultimately assigned are observed as relevant, whereas other suitable reviewers

in the candidate pool are not explicitly judged. Human expert assessment of additional paper-reviewer pairs would be required to construct a more complete expertise-based gold standard.

Papers without historical assignments, that is, papers for which

$$R_d^{\text{hist}} = \emptyset,$$

are excluded from aggregation. Prior to scoring, duplicate recommendations are removed by retaining only their first occurrence, and ranking ties are resolved deterministically using stable reviewer identifiers to ensure reproducibility.

A. EVALUATION METRICS

We use standard ranking-based metrics adapted to paper-reviewer matching. For a paper d , let R_d^{hist} denote its historical relevance set, and let $\pi_d(i)$ denote the reviewer ranked at position i . The metrics are computed independently for every paper and then macro-averaged over the set of evaluable papers $\mathcal{D}$. Per-paper metric values are retained so that method comparisons can be analyzed using paired statistical tests.

1) MEAN RECIPROCAL RANK (MRR@3)

Mean Reciprocal Rank measures how highly the *first* relevant reviewer appears and therefore places strong emphasis on the very top of the recommendation list.

Define the rank of the first relevant reviewer within the top-3 as:

$$r_d^{@K} := \min_{i \leq K: \pi_d(i) \in R_d},$$

with $r_d^{@K} = \infty$ if no relevant reviewer occurs in the top-3. The truncated reciprocal-rank score is then

$$\text{RR@K}(d) := \begin{cases} \frac{1}{r_d^{@K}}, & r_d^{@K} < \infty, \\ 0, & \text{otherwise.} \end{cases}$$

Finally, the macro-averaged metric is

$$\text{MRR@3} := \frac{1}{|\mathcal{D}|} \sum_{d \in \mathcal{D}} \text{RR@K}(d).$$

$\text{MRR@3} \in [0, 1]$, where larger values indicate that a relevant reviewer tends to appear earlier; $\text{MRR@3} = 1$ means a relevant reviewer is always ranked first.

2) PRECISION AT K (P@3)

Precision at K quantifies the fraction of the top-3 recommendations that are relevant, capturing the purity of the short-list.

For a paper d ,

$$\text{P@3}(d) := \frac{1}{K} \sum_{i=1}^K \mathbf{1}[\pi_d(i) \in R_d],$$

where $\mathbf{1}[\cdot]$ is the indicator function. The macro-average is

$$\text{P@3} := \frac{1}{|\mathcal{D}|} \sum_{d \in \mathcal{D}} \text{P@3}(d).$$

As with MRR@3 , $\text{P@3} \in [0, 1]$ and higher values are better.

3) MEAN AVERAGE PRECISION (MAP@3)

Mean Average Precision captures both *how many* relevant reviewers are retrieved and *how early* they appear, by averaging precision at the ranks where relevant reviewers occur.

Define precision at rank i for paper d as:

$$\text{P@i}(d) := \frac{1}{i} \sum_{j=1}^i \mathbf{1}[\pi_d(j) \in R_d].$$

Let $m_d = \min\{|R_d|, K\}$ be the maximum number of relevant reviewers that can contribute within the top-3. When a paper has more than K relevant reviewers, only the top-3 positions contribute to the score. The truncated average precision is

$$\text{AP@3}(d) := \begin{cases} \frac{1}{m_d} \sum_{i=1}^K \text{P@i}(d) \mathbf{1}[\pi_d(i) \in R_d], & m_d > 0, \\ 0, & m_d = 0. \end{cases}$$

The macro-averaged score is

$$\text{MAP@3} := \frac{1}{|\mathcal{D}|} \sum_{d \in \mathcal{D}} \text{AP@3}(d).$$

$\text{MAP@3} \in [0, 1]$ and rewards rankings that place relevant reviewers as high as possible while retrieving many of them.

B. EXPERIMENTS

Each paper is represented using one of three textual views: title, abstract summary, or full abstract. Each reviewer is represented by textual and keyword evidence aggregated from their prior publications. Given these representations, the system produces a ranked reviewer list using one of the evaluated lexical, neural, keyword-aware, or graph-based affinity functions.

Historical assignments define the observed relevance sets R_d^{hist} . Before evaluation, reviewer identifiers in both the recommendations and historical assignments are normalized through case folding, punctuation removal, and deduplication to ensure consistent matching. The same paper set, candidate-reviewer pool, relevance labels, identifier-normalization rules, and tie-breaking procedure are used across the methods being compared.

For each experimental configuration, we compute MRR@3 , P@3 , and MAP@3 and report macro-averages over all evaluable papers. To ensure fair comparisons among scoring functions, preprocessing, candidate pools, and deterministic ranking rules are held fixed.

When the contribution of a particular component is evaluated, only that component is changed. For example, comparisons of keyword-generation strategies retain the same downstream matcher and evaluation protocol, while comparisons of matching functions reuse the same precomputed keyword profiles. This controlled design reduces the risk that improvements caused by one pipeline stage are incorrectly attributed to another.

C. STATISTICAL SIGNIFICANCE AND CONFIDENCE INTERVALS

Because several methods produce similar aggregate ranking scores, we assess whether observed differences are supported by the corresponding per-paper results. For each document representation, metric, candidate method, and designated baseline, we compute the paired per-paper difference

$$\Delta_d = m_d^{\text{candidate}} - m_d^{\text{baseline}},$$

where m_d is the relevant per-paper value of reciprocal rank, Precision@ K , or average precision. Pairing is appropriate because the candidate and baseline methods are evaluated on the same papers and candidate-reviewer pools.

We apply a two-sided paired t -test to the vector of per-paper differences and report the mean difference, its 95% confidence interval, and the corresponding p -value. Confidence intervals are computed from the t distribution. When the parametric test cannot be reliably computed, for example because of a degenerate variance pattern, we use a paired percentile bootstrap with 10,000 resamples. The resampling unit is the paper, ensuring that candidate and baseline scores remain paired within every bootstrap sample.

We use $p < 0.05$ as the conventional significance threshold. However, statistical significance is interpreted jointly with the direction and magnitude of the mean difference and its confidence interval. Small differences whose intervals include zero are described as statistically indistinguishable rather than as confirmed improvements. The corresponding results are reported in Section VI.

D. WORKLOAD AND FAIRNESS METRICS

Beyond matching quality, a practical reviewer-assignment system must control reviewer load and avoid systematic over-assignment of a small subset of reviewers.

The workload analysis reported in this study is derived from the method-induced top-3 reviewer selections. Specifically, the top three reviewers returned for each paper are treated as provisional assignments for diagnostic purposes. These provisional assignments allow us to examine whether an affinity method repeatedly concentrates recommendations on the same reviewers. They do not constitute the output of a full capacity- and conflict-aware assignment optimizer.

Let $w_{r,c}$ denote the number of provisional assignments received by reviewer r in conference c . For each conference c , we restrict attention to the set of active reviewers,

$$R_c := \{r : w_{r,c} > 0\},$$

so that workload statistics are computed only over reviewers who receive at least one provisional assignment in that conference.

1) PER-CONFERENCE AVERAGE WORKLOAD

The mean number of assignments per active reviewer in conference c is [35]:

$$\bar{w}_c = \frac{1}{|R_c|} \sum_{r \in R_c} w_{r,c}.$$

This captures the typical load level within a conference but does not indicate how evenly that load is distributed.

2) MACRO-AVERAGE WORKLOAD ACROSS CONFERENCES

To summarize workloads uniformly across conferences, we compute the unweighted mean of the per-conference averages [54]:

$$\text{MacroAvg} = \frac{1}{|C'|} \sum_{c \in C'} \bar{w}_c,$$

where C' denotes the set of conferences in which at least one provisional assignment is made. This metric gives each conference equal influence regardless of size.

3) POOLED MEAN WORKLOAD

As a complementary aggregate that weights conferences by their number of active reviewers, we compute [54]:

$$\text{Pooled} = \frac{\sum_{c \in C'} \sum_{r \in R_c} w_{r,c}}{\sum_{c \in C'} |R_c|}.$$

Unlike MacroAvg, the pooled mean reflects the expected workload over all active reviewer–conference pairs.

4) GINI COEFFICIENT: WORKLOAD INEQUALITY

To quantify how unevenly provisional assignments are distributed among active reviewers within conference c , we use the Gini coefficient [55]. Let $\{x_i\}_{i=1}^n$ be the workload multiset

$$\{w_{r,c} : r \in R_c\},$$

where

$$n = |R_c| \quad \text{and} \quad \bar{x} = \frac{1}{n} \sum_{i=1}^n x_i = \bar{w}_c.$$

The Gini coefficient is

$$G_c = \frac{\sum_{i=1}^n \sum_{j=1}^n |x_i - x_j|}{2n^2 \bar{x}}.$$

The coefficient satisfies

$$G_c \in [0, 1],$$

where 0 indicates equal workloads among active reviewers and larger values indicate greater concentration of assignments.⁷

⁷Equivalently, the Gini coefficient is twice the area between the Lorenz curve and the line of perfect equality.

These statistics are intended as diagnostics of the workload tendencies induced by each affinity function. A method with strong ranking performance but a high Gini coefficient may repeatedly recommend a small set of broadly matched reviewers and may therefore require stronger capacity or fairness controls during final allocation.

E. INTEGRATION WITH COMPLETE REVIEWER ASSIGNMENT

The ranking metrics and provisional workload analysis evaluate the affinity-construction layer rather than a complete operational allocation system. In a complete framework, the full affinity matrix produced by a matcher is supplied to a downstream optimizer. Because complete conflict-of-interest and reviewer-capacity metadata were unavailable in the anonymized dataset, we provide an integration formulation and workload diagnostics rather than a full solver benchmark. Let a_{dr} denote the normalized affinity between paper d and reviewer r , and let $x_{dr} \in \{0, 1\}$ indicate whether reviewer r is assigned to paper d . A basic assignment objective is

$$\max_{\mathbf{x}} \sum_{d \in \mathcal{D}} \sum_{r \in \mathcal{R}} a_{dr} x_{dr}.$$

If paper d requires q_d reviewers, the allocation must satisfy

$$\sum_{r \in \mathcal{R}} x_{dr} = q_d, \quad \forall d \in \mathcal{D}.$$

Reviewer-capacity constraints can be represented as

$$\ell_r \leq \sum_{d \in \mathcal{D}} x_{dr} \leq u_r, \quad \forall r \in \mathcal{R},$$

where ℓ_r and u_r are the lower and upper assignment bounds for reviewer r . If $c_{dr} = 1$ denotes a known conflict of interest, conflicted assignments are excluded through

$$x_{dr} \leq 1 - c_{dr}, \quad \forall d, r.$$

Fairness can be incorporated through workload-balancing penalties, max–min paper-affinity objectives, topic-coverage constraints, or combinations of these mechanisms. The resulting optimization problem can be solved using integer programming, flow-based methods, Hungarian-style matching under simpler constraints, or specialized fairness-aware assignment algorithms.

The current experimental study does not compare alternative constrained assignment solvers. Instead, it isolates the affinity-estimation stage to determine how lexical, neural, keyword-aware, and graph-based representations affect reviewer ranking. The workload and Gini analyses provide assignment-side diagnostics but should not be interpreted as a substitute for an end-to-end capacity- and conflict-aware allocation experiment. A controlled comparison in which the evaluated affinity matrices are supplied to the same constrained optimizer represents an important extension of this work.

VI. RESULTS

This section evaluates reviewer–paper matching across three submission representations: paper titles, abstract summaries, and full abstracts. We compare lexical, neural, keyword-aware, re-ranking, and knowledge-graph-based affinity estimators. The lexical configurations include TF–IDF cosine similarity, keyword-set Jaccard similarity, and their unweighted average. The neural configurations include SBERT-based text similarity, keyword-aware representations, and a two-stage bi-encoder/cross-encoder pipeline. The graph-based configurations include Node2Vec embeddings and node- and edge-overlap estimators.

The experiments are organized around four research questions. **RQ1** examines how the submission representation title, summary, or abstract affects reviewer retrieval. **RQ2** compares lexical, neural, keyword-aware, re-ranking, and graph-based affinity estimators under a common evaluation protocol. **RQ3** examines the contribution of the selected LLM-generated keyword representation relative to text-only and non-LLM keyword representations. **RQ4** examines the workload tendencies induced by the resulting top-three recommendation lists and explains how the affinity scores can be integrated into a downstream constrained assignment framework.

To avoid treating the evaluated configurations as unrelated trials, we vary one principal component at a time. In field-level comparisons, the affinity estimator is held fixed while the submission representation is changed. In method-level comparisons, the representation and candidate reviewer pool are held fixed while the affinity estimator is changed. All keyword-dependent matchers consume the same precomputed, judge-selected keyword profiles for the corresponding representation; the producer and judge prompts are not tuned separately for individual matching methods.

We report MRR@3, P@3, and MAP@3 according to the evaluation protocol defined in Section V. Numerical ordering is reported descriptively, while claims of statistical superiority are reserved for comparisons supported by the uncertainty analysis in Section V-C.

A. TITLE-ONLY RETRIEVAL: OVERALL TRENDS

Table 1 reports title-based retrieval performance across lexical, neural, keyword-aware, re-ranking, and graph-based configurations. The results show different operating points for first-hit ranking and shortlist density. The Keyword-Aware Two-Stage model obtains the numerically highest MRR@3 (0.7778), whereas Keyword-Aware SBERT obtains the highest P@3 (0.4359). Average Cosine and Jaccard obtains the numerically highest MAP@3 (0.2993), only 0.0001 above Keyword-Aware SBERT (0.2992). Because this difference is extremely small, the two methods should be regarded as practically comparable on MAP@3 unless the paired uncertainty analysis establishes otherwise. Overall, the title results indicate that re-ranking, keyword-aware semantic matching, and lexical–semantic fusion emphasize different

TABLE 1. Reviewer-matching performance using paper titles.

	Title		
	MRR@3	P@3	MAP
Average Cosine and Jaccard	0.7157	0.3922	0.2993
Overlapping (Jaccard)	0.6892	0.3649	0.2639
TF-IDF (Cosine)	0.7078	0.3882	0.2941
Keyword-Aware Bi-Encoder (Cosine)	0.6605	0.3853	0.2657
Keyword-Aware Two-Stage (Bi-Encoder + Cross-Encoder)	0.7778	0.3720	0.2978
SBERT (Cosine)	0.6571	0.3873	0.2636
Keyword-Aware SBERT (Cosine)	0.6025	0.4359	0.2992
Hybrid SBERT (Title + Keywords)	0.6625	0.3875	0.2666
Adaptive Fusion (Text + Soft Keywords)	0.6634	0.3949	0.2743
Soft Keyword Coverage (IDF-weighted)	0.7154	0.3780	0.2825
SBERT Text Similarity baseline	0.6997	0.3961	0.2871
KG Edge Jaccard Similarity	0.7556	0.3556	0.2741
KG Node Jaccard (1-hop)	0.5952	0.3470	0.2193
Node2Vec (Cosine)	0.7349	0.3775	0.2888
Hybrid Text-KG (Cosine + Node Jaccard)	0.6749	0.3621	0.2544
Local KG (Node2Vec-Cosine + Jaccard)	0.6875	0.3694	0.2395
Local KG (Jaccard)	0.6818	0.3637	0.2615
Local KG (Node2Vec-Cosine)	0.6250	0.3533	0.2187

parts of the top-three ranking rather than identifying one configuration that is uniformly best across all metrics.

1) LEXICAL BASELINES AND LEXICAL-SEMANTIC FUSION

Lexical matching remains highly competitive in the title-only setting. TF-IDF (Cosine) achieves strong performance (0.7078/0.3882/0.2941 for MRR@3/P@3/MAP), confirming that discriminative title terms are sufficiently informative when weighted by rarity. Pure overlap (Jaccard) is weaker overall (0.6892/0.3649/0.2639), consistent with its inability to account for partial semantic similarity or term importance. Notably, the fusion Average Cosine and Jaccard improves over Jaccard by +0.0265 MRR@3 (+3.85%), +0.0273 P@3 (+7.48%), and +0.0354 MAP (+13.41%), and also provides a consistent gain over TF-IDF by +0.0079 MRR@3 (+1.12%), +0.0040 P@3 (+1.03%), and +0.0052 MAP (+1.77%). This pattern suggests that, for titles, an explicit exact-overlap channel complements cosine similarity by reinforcing shared topical anchors while TF-IDF (or cosine-based scoring) captures graded similarity and term salience.

2) NEURAL SEMANTIC SIMILARITY AND THE ROLE OF KEYWORD AWARENESS

Among embedding-based text methods, SBERT (Cosine) obtains 0.6571/0.3873/0.2636 for MRR@3/P@3/MAP@3, implying that general-purpose sentence embeddings do not consistently dominate when queries and candidates are short titles dominated by a small set of specialized nouns. The SBERT Text Similarity baseline is comparatively better balanced (0.6997/0.3961/0.2871), improving on SBERT by +0.0426 MRR@3 (+6.48%) and +0.0235 MAP (+8.89%), but it still trails the best MAP achieved by lexical-semantic fusion.

Keyword conditioning substantially changes the operating point of neural similarity. Keyword-Aware SBERT (Cosine) attains the best P@3 (0.4359) and essentially matches the best MAP (0.2992), but its MRR@3 drops to 0.6025. Relative

to SBERT, this corresponds to a large increase in shortlist precision (+0.0486 P@3, +12.55%) and MAP (+0.0356, +13.48%), alongside a decrease in first-hit quality (−0.0546 MRR@3, −8.31%).

This operating-point shift suggests that the selected keyword representation is associated with greater relevant-item density in the top-three list, while not improving first-hit ranking in this configuration. Hybrid SBERT and Adaptive Fusion obtain lower MAP@3 values than Keyword-Aware SBERT; however, these numerical differences should not be interpreted as evidence about the intrinsic superiority of one keyword-integration mechanism without the corresponding paired statistical comparisons.

3) TWO-STAGE RE-RANKING: OPTIMIZING RANK-1 PERFORMANCE

The two-stage pipeline delivers the strongest early-rank performance. Compared to its corresponding Keyword-Aware Bi-Encoder, adding cross-encoder re-ranking increases MRR@3 by +0.1173 (+17.76%) and MAP by +0.0321 (+12.08%), while slightly reducing P@3 by −0.0133 (−3.45%). When compared to the best MAP method (Average Cosine and Jaccard), the two-stage approach improves MRR@3 by +0.0621 (+8.68%) but decreases P@3 by −0.0202 (−5.15%) and yields a marginally lower MAP (−0.0015, −0.50%). This behavior is characteristic of re-ranking models that sharpen the top of the list (benefiting MRR@3) without necessarily increasing the density of relevant items throughout the top-3 (which affects P@3 and MAP).

4) KNOWLEDGE-GRAPH SIMILARITY: STRONG EARLY SIGNALS, WEAKER LIST QUALITY

KG-based methods provide competitive early hits but generally weaker shortlist precision and cumulative ranking quality than the strongest text-based approaches. KG Edge Jaccard Similarity achieves very high MRR@3 (0.7556,

second-best overall) but a low P@3 (0.3556), indicating that structural overlap can surface an early correct match but does not reliably populate the top-3 with multiple relevant items. Node2Vec (Cosine) is the strongest KG embedding baseline (0.7349/0.3775/0.2888), yet it remains below the best MAP methods, suggesting that title text carries more direct relevance evidence than graph proximity alone.

The local-KG variants obtain lower reported values than the global Node2Vec configuration. For example, the difference between Node2Vec and Local KG (Node2Vec–Cosine) is +0.1099 in MRR@3 and +0.0701 in MAP@3. These local configurations should be interpreted as heuristic baselines because their graph embeddings are constructed from document-local structures; their lower performance should not be attributed to sparsity alone without a controlled graph-construction ablation.

5) IMPLICATIONS FOR TITLE-BASED MATCHING

Under the current evaluation protocol, Average Cosine and Jaccard obtains the numerically highest MAP@3 for titles, the Keyword-Aware Two-Stage model obtains the highest MRR@3, and Keyword-Aware SBERT obtains the highest P@3. These results identify different operating points rather than a single universally superior method. The preferred configuration therefore depends on whether the application prioritizes the first historically assigned reviewer, the density of such reviewers in the top-three shortlist, or cumulative top-three ranking quality. Because several reported differences are small, statistical uncertainty must be considered before interpreting the numerical ordering as evidence of superiority.

B. SUMMARY-ONLY RETRIEVAL: OVERALL TRENDS

Table 2 reports reviewer-matching performance when papers are represented by abstract summaries. Keyword-Aware SBERT obtains the numerically highest P@3 (0.4545) and MAP@3 (0.3232), whereas the Keyword-Aware Two-Stage model obtains the highest MRR@3 (0.8000). The results therefore again reveal a distinction between optimizing the first relevant result and increasing the density of relevant reviewers in the top-three shortlist. Compared with titles, summaries provide more semantic context, and the strongest reported list-quality values are obtained by a keyword-aware semantic representation. This observation is descriptive; whether the differences are statistically reliable is examined separately in Section V-C.

1) LEXICAL BASELINES AND LEXICAL–SEMANTIC FUSION

Lexical matching remains competitive but no longer dominates on summaries. TF–IDF (Cosine) achieves strong performance (0.6944/0.3838/0.2820 for MRR@3/P@3/MAP), indicating that term-weighted overlap still captures relevant topical cues even in longer text. The fusion Average Cosine and Jaccard provides a similar operating point (0.7128/0.3795/0.2889), improving over pure Jaccard overlap (0.6442/0.3462/0.2382) by +0.0686 MRR@3 (+10.65%), +0.0333 P@3 (+9.62%), and +0.0507

MAP (+21.29%). However, neither TF–IDF nor the cosine–Jaccard fusion approaches the MAP achieved by Keyword-Aware SBERT, suggesting that the additional semantic variability in summaries reduces the effectiveness of purely lexical similarity compared to methods that can align paraphrases and abstract phrasing.

2) NEURAL SEMANTIC SIMILARITY AND THE IMPACT OF KEYWORD AWARENESS

Among purely semantic baselines, SBERT (Cosine) reaches 0.6858/0.3878/0.2772, while the SBERT Text Similarity baseline provides a slightly stronger balance (0.6895/0.3954/0.2873). The most significant gains appear when keyword signals are injected directly into the representation: Keyword-Aware SBERT yields 0.6212/0.4545/0.3232, producing the best P@3 and MAP across all methods. Relative to SBERT (Cosine), this corresponds to a large increase of +0.0667 in P@3 (+17.19%) and +0.0460 in MAP (+16.59%), at the cost of reduced MRR@3 (−0.0646, −9.42%). This shift indicates that keyword-aware semantic similarity is particularly effective at retrieving a larger set of relevant candidates from longer summaries, even if the single top-ranked item is not consistently improved. The weaker MRR@3 suggests that keyword emphasis may broaden retrieval toward thematically related papers, increasing shortlist density but occasionally diluting top-1 precision.

Hybrid SBERT (Summary + Keywords) and Adaptive Fusion do not match the gains of Keyword-Aware SBERT. Hybrid SBERT reaches 0.6778/0.3600/0.2514, and Adaptive Fusion reaches 0.6336/0.3836/0.2544, both underperforming SBERT baselines in MAP. This indicates that simply concatenating keywords or blending them weakly may introduce noise or mismatched emphasis in longer summary text; stronger conditioning mechanisms that reshape the embedding space appear more effective than surface-level fusion in this setting.

3) TWO-STAGE RE-RANKING: STRONGEST RANK-1 PERFORMANCE

The Keyword-Aware Two-Stage model achieves the best rank-1 behavior with MRR@3 = 0.8000, exceeding the best non-reranked baselines (e.g., Average Cosine and Jaccard at 0.7128) by +0.0872 (+12.24%). Its MAP@3 = 0.3016 is also high, but it remains below Keyword-Aware SBERT's MAP by −0.0216 (−6.68%). Additionally, its P@3 = 0.3667 is substantially lower than the keyword-aware SBERT shortlist precision. This pattern is consistent with cross-encoder re-ranking strongly improving the very top of the list by modeling fine-grained interactions between query and candidate summaries, while not necessarily increasing the number of relevant items distributed throughout the top-3 set.

4) KNOWLEDGE-GRAPH SIMILARITY ON SUMMARIES

KG-based methods remain weaker overall than the strongest summary-text approaches, though they can provide competitive early hits. KG Edge Jaccard achieves high

TABLE 2. Reviewer-matching performance using abstract summaries.

	Summary		
	MRR@3	P@3	MAP
Average Cosine and Jaccard	0.7128	0.3795	0.2889
Overlapping (Jaccard)	0.6442	0.3462	0.2382
TF-IDF (Cosine)	0.6944	0.3838	0.2820
Keyword-Aware Bi-Encoder (Cosine)	0.6842	0.3910	0.2785
Keyword-Aware Two-Stage (Bi-Encoder + Cross-Encoder)	0.8000	0.3667	0.3016
SBERT (Cosine)	0.6858	0.3878	0.2772
Keyword-Aware SBERT (Cosine)	0.6212	0.4545	0.3232
Hybrid SBERT (Summary + Keywords)	0.6778	0.3600	0.2514
Adaptive Fusion (Text + Soft Keywords)	0.6336	0.3836	0.2544
Soft Keyword Coverage (IDF-weighted)	0.6309	0.3669	0.2460
SBERT Text Similarity baseline	0.6895	0.3954	0.2873
KG Edge Jaccard Similarity	0.7639	0.3478	0.2650
KG Node Jaccard (1-hop)	0.6264	0.3455	0.2203
Node2Vec (Cosine)	0.6186	0.3615	0.2420
Hybrid Text-KG (Cosine + Node Jaccard)	0.6101	0.3630	0.2385
Local KG (Node2Vec-Cosine + Jaccard)	0.5833	0.3760	0.1945
Local KG (Jaccard)	0.6289	0.3459	0.2311
Local KG (Node2Vec-Cosine)	0.5476	0.3593	0.1825

MRR@3 (0.7639) but low P@3 (0.3478) and MAP (0.2650), indicating that structural overlap can surface a plausible match early but does not yield strong overall ranking quality for summary-based retrieval. Node2Vec (Cosine) and hybrid text-KG variants underperform substantially (e.g., Node2Vec at 0.6186/0.3615/0.2420; Hybrid Text-KG at 0.6101/0.3630/0.2385), suggesting that summary semantics provide a much richer relevance signal than graph proximity alone, and that simple additive combinations with shallow neighborhood overlap are insufficient to exploit complementary structure. Local KG variants are the weakest overall (e.g., Local KG Node2Vec-Cosine at MAP@3 = 0.1825), reinforcing that restricting to local neighborhoods likely exacerbates sparsity and limits the ability to capture broader topical similarity.

5) IMPLICATIONS FOR SUMMARY-BASED MATCHING

For summaries, Keyword-Aware SBERT obtains the numerically highest P@3 and MAP@3, while the Keyword-Aware Two-Stage pipeline obtains the highest MRR@3. This pattern suggests that keyword-aware semantic representation and cross-encoder re-ranking emphasize different parts of the ranked list. The findings should therefore be interpreted as a trade-off between shortlist density and first-hit ranking, rather than as evidence that one architecture dominates every evaluation objective. The lexical configurations remain competitive, but their numerical ordering relative to the keyword-aware methods should be interpreted together with the paired confidence intervals and adjusted *p*-values.

C. ABSTRACT-ONLY RETRIEVAL: OVERALL TRENDS

Table 3 reports reviewer-matching performance using full abstracts. KG Edge Jaccard Similarity obtains the numerically highest MRR@3 (0.7843) and MAP@3 (0.2991). Keyword-Aware SBERT obtains the highest P@3 (0.4000), while the SBERT Text Similarity baseline provides the strongest balance among the text-only configurations, with

an MRR@3 of 0.7152, a P@3 of 0.3958, and a MAP@3 of 0.2922. Thus, no single configuration dominates all three evaluation metrics. The reported values suggest that semantic text similarity, keyword-aware matching, and graph-edge overlap provide different advantages for abstract-based reviewer retrieval; however, the statistical reliability of the observed differences is assessed separately in the significance analysis.

1) LEXICAL BASELINES: LIMITED GAINS FROM TERM OVERLAP ON ABSTRACTS

Pure lexical methods are comparatively weaker on abstracts than on titles. Average Cosine and Jaccard reach 0.6379/0.3704/0.2476 (MRR@3/P@3/MAP), while TF-IDF (Cosine) achieves 0.6235/0.3686/0.2435. Notably, Jaccard overlap slightly exceeds the fusion baseline on MRR@3 (0.6505 vs. 0.6379) but remains below it on P@3 and MAP, suggesting that abstracts contain more paraphrastic and descriptive language where exact overlap alone is insufficient, yet shared surface terms can still occasionally place a relevant abstract early. Overall, the narrow spread among lexical methods (MAP $\approx$ 0.243–0.248) indicates diminishing returns from purely term-based similarity once the text becomes longer and more varied.

2) NEURAL SEMANTIC SIMILARITY AND KEYWORD INTEGRATION

Semantic embedding methods provide a substantial improvement over lexical baselines. SBERT (Cosine) achieves 0.7023/0.3911/0.2857, improving MAP by approximately +0.038 relative to Average Cosine and Jaccard. The SBERT Text Similarity baseline obtains 0.7152/0.3958/0.2922 for MRR@3/P@3/MAP@3 and provides the strongest balance among the text-only configurations. Its P@3 is slightly below that of Keyword-Aware SBERT (0.4000), which is the highest reported P@3 in the abstract experiment.

TABLE 3. Reviewer-matching performance using full paper abstracts.

	Abstract		
	MRR@3	P@3	MAP
Average Cosine and Jaccard	0.6379	0.3704	0.2476
Overlapping (Jaccard)	0.6505	0.3548	0.2437
TF-IDF (Cosine)	0.6235	0.3686	0.2435
Keyword-Aware Bi-Encoder (Cosine)	0.6846	0.3889	0.2778
Keyword-Aware Two-Stage (Bi-Encoder + Cross-Encoder)	0.7412	0.3731	0.2869
SBERT (Cosine)	0.7023	0.3911	0.2857
Keyword-Aware SBERT (Cosine)	0.6500	0.4000	0.2805
Hybrid SBERT (Abstract + Keywords)	0.6729	0.3791	0.2604
Adaptive Fusion (Text + Soft Keywords)	0.6748	0.3856	0.2736
Soft Keyword Coverage (IDF-weighted)	0.6244	0.3755	0.2528
SBERT Text Similarity baseline	0.7152	0.3958	0.2922
KG Edge Jaccard Similarity	0.7843	0.3689	0.2991
KG Node Jaccard (1-hop)	0.6758	0.3461	0.2455
Node2Vec (Cosine)	0.6988	0.3784	0.2559
Hybrid Text-KG (Cosine + Node Jaccard)	0.6832	0.3609	0.2251
Local KG (Node2Vec-Cosine + Jaccard)	0.6667	0.3785	0.2350
Local KG (Jaccard)	0.6416	0.3611	0.2445
Local KG (Node2Vec-Cosine)	0.5889	0.3601	0.2074

Keyword-aware variants produce moderate but not dominant gains in this setting. Keyword-Aware Bi-Encoder improves over lexical methods (0.6846/0.3889/0.2778), but remains slightly below SBERT (Cosine) in both MRR@3 and MAP. Keyword-Aware SBERT increases P@3 to 0.4000 and maintains a strong MAP (0.2805), but it does not surpass the SBERT text similarity baseline or the graph-based best method. This suggests that the abstract text itself already contains many of the concepts that keywords would contribute; consequently, keyword conditioning has less leverage than in summaries, where keywords more strongly act as a relevance prior.

Among fusion strategies, Adaptive Fusion (Text + Soft Keywords) reaches 0.6748/0.3856/0.2736, improving on Hybrid SBERT (Abstract + Keywords) (0.6729/0.3791/0.2604) and the soft keyword coverage baseline (0.6244/0.3755/0.2528). This indicates that softly integrating keyword evidence is more robust than direct concatenation, but the overall gains remain smaller than those achieved by stronger semantic modeling.

3) TWO-STAGE RE-RANKING: IMPROVING EARLY PRECISION

The Keyword-Aware Two-Stage (Bi-Encoder + Cross-Encoder) model yields strong early-rank performance (MRR@3 = 0.7412) with a competitive MAP (0.2869). However, it does not exceed the SBERT Text Similarity baseline on MAP nor the KG Edge Jaccard method on either MRR@3 or MAP. Its P@3 (0.3731) also remains below the top text-based performers. This outcome suggests that, while cross-encoder re-ranking improves fine-grained matching between abstract pairs, the abstract setting already offers enough semantic signal for the SBERT Text Similarity baseline to produce strong lists, and the KG structural signal provides an additional advantage that re-ranking alone does not replicate.

4) KNOWLEDGE-GRAPH METHODS ON ABSTRACTS

KG Edge Jaccard obtains the numerically highest MRR@3 (0.7843) and MAP@3 (0.2991) in the abstract experiment. Compared with SBERT (Cosine), its reported values are higher by 0.0820 in MRR@3 and 0.0134 in MAP@3. These numerical differences suggest that edge-overlap information may provide complementary evidence at the abstract level; however, the interpretation should be conditioned on the paired uncertainty analysis.

Node2Vec (Cosine) performs reasonably (0.6988/0.3784/0.2559) but remains well below KG Edge Jaccard in MRR@3 and MAP, suggesting that direct neighborhood/edge-overlap similarity is more effective than purely embedding-based graph proximity for this dataset. Hybrid Text-KG (Cosine + Node Jaccard) performs poorly in MAP (0.2251), implying that a naive additive combination may introduce noise or miscalibrate ranking, particularly when one component (Node Jaccard) is weakly aligned with abstract-level relevance. Local KG variants are also consistently weaker, with Local KG (Node2Vec-Cosine) producing the lowest MAP (0.2074), reinforcing that restricting graph context likely harms robustness.

5) IMPLICATIONS FOR ABSTRACT-BASED MATCHING

For abstracts, KG Edge Jaccard obtains the numerically highest MRR@3 and MAP@3, while Keyword-Aware SBERT obtains the highest P@3. The SBERT Text Similarity baseline remains the strongest balanced text-only configuration. These findings identify complementary operating points for semantic text matching, keyword-aware retrieval, and graph-based overlap. They should not be interpreted as evidence of a universally superior method unless the corresponding paired confidence interval excludes zero and the multiplicity-adjusted p -value supports the comparison.

D. OVERALL COMPARISON ACROSS FIELDS

Figure 2 provides a compact comparison of the reported methods across titles, summaries, and abstracts. The numerical ordering depends on both the submission representation and the evaluation objective. For titles, Average Cosine and Jaccard obtains the highest MAP@3, the Keyword-Aware Two-Stage model obtains the highest MRR@3, and Keyword-Aware SBERT obtains the highest P@3. For summaries, Keyword-Aware SBERT obtains the highest P@3 and MAP@3, while the two-stage model again obtains the highest MRR@3. For abstracts, KG Edge Jaccard obtains the highest MRR@3 and MAP@3, whereas Keyword-Aware SBERT obtains the highest P@3.

A recurring distinction is therefore observed between methods that prioritize the first relevant reviewer and methods that increase the density of relevant reviewers in the top three shortlist. The two-stage model obtains the highest reported MRR@3 for titles and summaries, while Keyword-Aware SBERT obtains the highest reported P@3 for all three representations. These patterns motivate a layered architecture in which a high-recall retriever generates a candidate shortlist and a more computationally intensive re-ranker refines the ordering when first hit quality is operationally important. However, the architecture should be selected using the intended assignment objective rather than numerical ranking alone, because several differences are small and may not remain statistically distinguishable after correction for multiple comparisons.

E. EXTERNAL BASELINES AND PARTIAL LLM-COMPONENT ABLATION

To address the contribution of the LLM keyword-induction component more directly, we compare the proposed framework against external baselines and a single-LLM reference configuration. The previous lexical baseline results in Table 4 are taken from our earlier content-based reviewer-assignment study [50], which evaluated cosine similarity, dot-product similarity, and Jaccard similarity over titles, summaries, and abstracts. Table 5 reports GPT-4o-mini as a single-LLM keyword-generation reference, while Table 6 reports BM25, KeyBERT, and SPECTER2 baselines from our LLM reviewer-matching study [51].

These comparisons should be interpreted as an external-baseline and partial component ablation rather than as a complete producer-by-producer ablation. In particular, KeyBERT provides a non-generative keyword-extraction reference, BM25 provides a lexical retrieval reference, SPECTER2 provides a scientific-document embedding reference, and GPT-4o-mini provides a single-LLM keyword-generation reference. This allows us to quantify whether the proposed ensemble-and-judge framework provides benefits beyond simpler lexical, embedding-based, and single-model alternatives.

The external baselines show that simple lexical and non-generative keyword-based methods remain competitive in

TABLE 4. Previous lexical reviewer-matching baselines reported in our earlier content-based reviewer-assignment study [50]. Values correspond to the constrained maximum-similarity setting and are used here as external non-LLM baselines.

Text View	Similarity Method	MRR	MAP@3	P@3
Abstract	Cosine Similarity	0.7054	0.4018	0.3196
Abstract	Dot Product Similarity	0.6452	0.3817	0.3118
Abstract	Jaccard Similarity	0.4929	0.5301	0.3216
Summary	Cosine Similarity	0.3974	0.2307	0.1871
Summary	Dot Product Similarity	0.5833	0.3541	0.3125
Summary	Jaccard Similarity	0.2923	0.1217	0.0840
Title	Cosine Similarity	0.4444	0.2546	0.2000
Title	Dot Product Similarity	0.6904	0.3928	0.2346
Title	Jaccard Similarity	0.1944	0.1157	0.1083

TABLE 5. Single-LLM keyword-generation reference using GPT-4o-mini, reported in our LLM reviewer-matching study [51].

Text View	Similarity Method	MRR	MAP@3	P@3
Title	Cosine-Jaccard Average Similarity	0.6666	0.2624	0.3650
Title	Cosine Similarity	0.6650	0.2576	0.3627
Title	Jaccard Similarity	0.6255	0.2541	0.3703
Summary	Cosine-Jaccard Average Similarity	0.6279	0.2567	0.3720
Summary	Cosine Similarity	0.6352	0.2633	0.3764
Summary	Jaccard Similarity	0.6395	0.2586	0.3749
Abstract	Cosine-Jaccard Average Similarity	0.6524	0.2398	0.3495
Abstract	Cosine Similarity	0.6687	0.2492	0.3544
Abstract	Jaccard Similarity	0.7229	0.2759	0.3603

some settings, but their behavior differs from the proposed ensemble-and-judge framework. The previous lexical baselines achieve strong results in specific constrained settings, particularly for title-based dot-product similarity and abstract-based Jaccard similarity. However, these approaches rely on direct lexical or vector-space matching and do not provide the same LLM-generated, role-specific keyword evidence used in the proposed framework.

The GPT-4o-mini reference provides a single-LLM comparison for the keyword-generation component. It performs strongly, especially for abstract-based Jaccard similarity, but it represents only one producer model. In contrast, the proposed ensemble-and-judge architecture uses heterogeneous producers and a separate judge to select the most faithful and specific candidate keyword list. Therefore, the comparison indicates that single-model keyword generation is a strong baseline, while the ensemble design is intended to reduce dependence on a single model's terminology, verbosity, and failure modes.

The BM25, KeyBERT, and SPECTER2 results further contextualize the proposed method. BM25 is competitive for title-based lexical retrieval and obtains the highest P@3 among the abstract baselines, while SPECTER2 performs strongly for summary and abstract MAP@3. KeyBERT provides a useful non-generative keyword baseline, but its MAP@3 values remain below the strongest SPECTER2 and BM25 configurations in these external comparisons. Overall, these results strengthen the empirical positioning of the proposed framework by showing that it is compared not only with internal matching variants but also with prior lexical baselines, a single-LLM keyword baseline, a non-generative keyword extractor, a lexical retrieval baseline, and a scientific-document embedding baseline.



FIGURE 2. Retrieval performance across methods and submission representations. Each cell reports the corresponding MRR@3, P@3, or MAP@3 value. Background shading is based on column-wise min-max normalization, and outlined cells denote the numerically highest value for each metric and representation.

TABLE 6. External baseline reviewer-matching performance using BM25, KeyBERT, and SPECTER2. BM25 provides a lexical retrieval baseline, KeyBERT provides a non-generative keyword-extraction baseline, and SPECTER2 provides a scientific-document embedding baseline.

Text View	Method	Configuration	MRR	P@3	MAP@3
Title	BM25	Lexical Similarity	0.6013	0.2766	0.2074
Title	KeyBERT	Cosine-Jaccard Average Similarity	0.5694	0.2710	0.1751
Title	KeyBERT	Cosine Similarity	0.5719	0.2698	0.1743
Title	KeyBERT	Jaccard Similarity	0.5424	0.2750	0.1523
Title	SPECTER2	Embedding-Based Similarity	0.5174	0.2756	0.1488
Summary	BM25	Lexical Similarity	0.5400	0.2733	0.1678
Summary	KeyBERT	Cosine-Jaccard Average Similarity	0.5111	0.2667	0.1389
Summary	KeyBERT	Cosine Similarity	0.5022	0.2656	0.1348
Summary	KeyBERT	Jaccard Similarity	0.5991	0.2611	0.1654
Summary	SPECTER2	Embedding-Based Similarity	0.5687	0.2943	0.1907
Abstract	BM25	Lexical Similarity	0.4913	0.2790	0.1374
Abstract	KeyBERT	Cosine-Jaccard Average Similarity	0.5464	0.2649	0.1470
Abstract	KeyBERT	Cosine Similarity	0.5513	0.2640	0.1500
Abstract	KeyBERT	Jaccard Similarity	0.5111	0.2556	0.1296
Abstract	SPECTER2	Embedding-Based Similarity	0.5378	0.2782	0.1730

F. CONTRIBUTION OF THE SELECTED KEYWORD REPRESENTATION

The existing configurations permit a controlled comparison of text-only and keyword-aware variants within the SBERT family, although they do not permit reconstruction of every individual producer model. For titles, Keyword-Aware SBERT changes the reported values from 0.6571/0.3873/0.2636 for SBERT to 0.6025/0.4359/0.2992 for MRR@3/P@3/MAP@3. Thus, P@3 and MAP@3 increase, while MRR@3 decreases. For summaries, the corresponding change is from 0.6858/0.3878/0.2772 to 0.6212/0.4545/0.3232, again shifting the operating point toward higher shortlist density and MAP@3 but lower MRR@3. For abstracts, Keyword-Aware SBERT increases

P@3 from 0.3911 to 0.4000, while its MRR@3 and MAP@3 remain below the corresponding SBERT values.

These comparisons suggest that the selected keyword profiles primarily shift the SBERT operating point toward greater top-three shortlist density, particularly for titles and summaries, rather than producing a uniform improvement across all metrics. This analysis isolates the effect of supplying the selected keyword representation within the same broad encoder family. It does not establish that the ensemble-and-judge procedure outperforms every individual producer or every traditional keyword-extraction method.

The original artifacts retain the judge-selected keyword list and the identity of the winning producer, but not all non-selected candidate lists. Consequently,

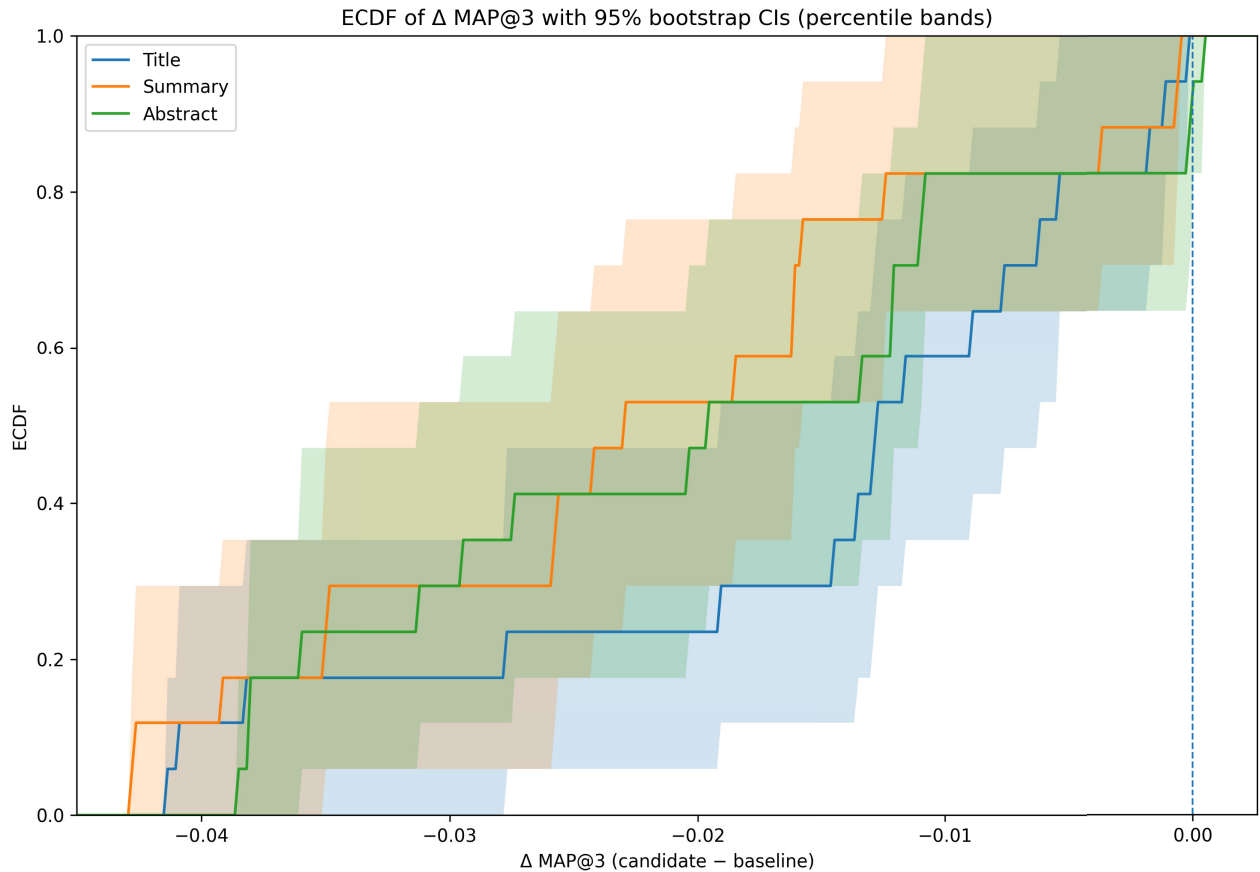


FIGURE 3. Empirical cumulative distributions of MAP@3 differences between candidate methods and the designated baseline for titles, summaries, and abstracts. Shaded regions denote 95% percentile bootstrap confidence bands based on 5,000 resamples. The vertical line at $\Delta = 0$ denotes numerical parity.

individual-producer reviewer rankings cannot be reconstructed retrospectively. We therefore treat the winner-model analysis as a diagnostic of judge behavior rather than as a producer-level performance ablation.

Significance Testing: To assess whether the observed gaps are statistically reliable, we ran paired two-sided t -tests over per-paper deltas (candidate–baseline) for each metric and representation, with confidence intervals (CIs) for the mean difference computed from the t distribution; when a t -test could not be computed, we fell back to a paired percentile bootstrap (10,000 resamples). This paired design controls for instance-level variance across the same set of papers and reviewer pools. Throughout, we report Δ (candidate–baseline) with 95% CIs and treat $p < 0.05$ as statistically significant.

Overall, many method–baseline differences are small and statistically indistinguishable from zero, especially for standard text encoders. For example, on titles, Keyword-Aware Bi-Encoder (Cosine) versus the SBERT Text Similarity baseline shows a negligible change in P@3 (with $K = 3$): $\Delta P@3 = +0.0030$ with 95% CI $[-0.0022, 0.0082]$ ($p = 0.257$), and similarly small effects on MAP@3 ($\Delta = -0.00017$, 95% CI $[-0.00292, 0.00259]$, $p = 0.905$) and MRR@3 ($\Delta = +0.00201$, 95% CI $[-0.00521, 0.00923]$,

$p = 0.588$). On summaries, SBERT (Cosine) essentially matches the baseline: $\Delta P@3 = 0.000$ with 95% CI $[-0.00370, 0.00370]$ ($p = 1.0$), $\Delta MAP@3 = -0.00071$ with 95% CI $[-0.00334, 0.00192]$ ($p = 0.595$), and $\Delta MRR@3 = +0.00151$ with 95% CI $[-0.00429, 0.00731]$ ($p = 0.610$). On abstracts, Adaptive Fusion (Text + Soft Keywords) remains close to the baseline as well (e.g., $\Delta P@3 = +0.00201$, 95% CI $[-0.00496, 0.00898]$, $p = 0.527$).

In contrast, several knowledge-graph and local-KG configurations exhibit clear degradations where 95% CIs do not straddle zero. For instance, on titles, Local KG (Node2Vec–Cosine) significantly underperforms the baseline on MAP@3: $\Delta MAP@3 = -0.0414$ with 95% CI $[-0.0507, -0.0321]$ ($p = 1.64 \times 10^{-17}$).

We also observe borderline effects near the 0.05 threshold (e.g., Soft Keyword Coverage (IDF-weighted) on titles for MAP@3: $\Delta = -0.0090$, 95% CI $[-0.0181, 0.00021]$, $p = 0.055$), suggesting small but not conventionally significant disadvantages at the 95% level.

To complement the method-level paired tests, we visualize the empirical distribution of MAP@3 differences relative to the baseline in Figure 3. For each representation, the ECDF shows how the candidate–baseline differences are

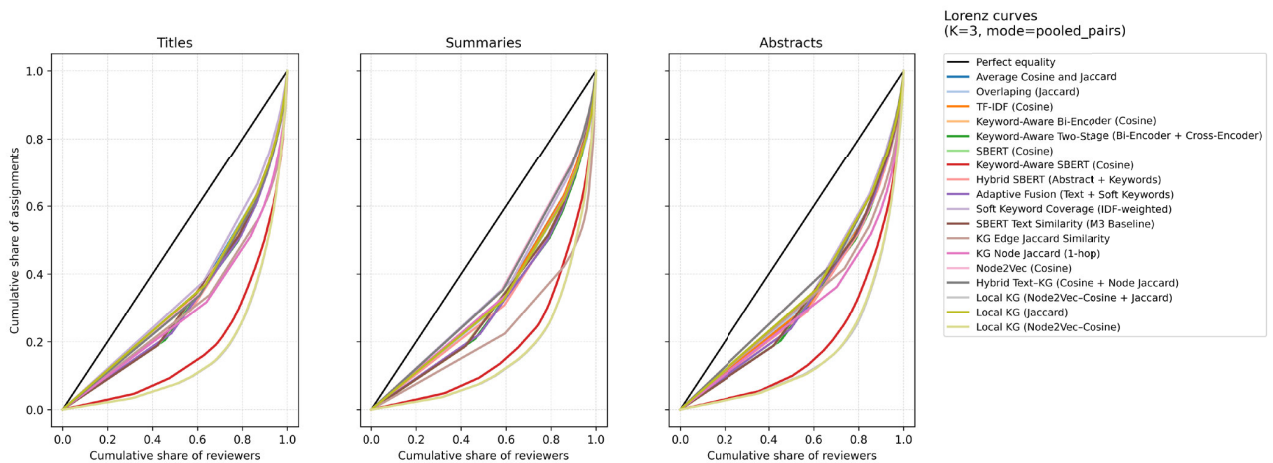


FIGURE 4. Lorenz curves of provisional reviewer-conference workloads induced by unconstrained top-three recommendation lists for titles, summaries, and abstracts.

distributed across the evaluated configurations. The shaded regions denote 95% percentile bootstrap confidence bands obtained from 5,000 resamples. This figure is intended as a descriptive summary of the overall distribution of differences; statistical conclusions for individual methods are based on the paired comparisons reported in this subsection.

G. PROVISIONAL WORKLOAD DIAGNOSTICS FROM UNCONSTRAINED TOP-3 RANKINGS

In addition to ranking performance, we examine the workload tendencies induced by each method's unconstrained top three recommendation lists. For diagnostic purposes, each occurrence of a reviewer in a paper's top-three list is treated as one provisional recommendation. These counts are not generated by a global capacity- and conflict-aware assignment optimizer and should therefore not be interpreted as final conference assignments.

For each conference, we compute the mean number of provisional recommendations received by an active reviewer. We summarize this distribution in three ways: the macro-average workload, defined as the unweighted mean of the per-conference averages; the pooled mean workload, computed across all reviewer-conference pairs; and the Gini coefficient of reviewer-conference recommendation counts, where zero indicates an even distribution and larger values indicate greater concentration.

For the SBERT Text Similarity baseline, the induced workloads remain similar across representations. For titles, the macro-average is 1.846, the pooled mean is 2.305, and the reviewer-conference Gini coefficient is 0.375. For summaries, the corresponding values are 1.864, 2.315, and 0.376; for abstracts, they are 1.860, 2.315, and 0.373. These values indicate that the baseline recommendation lists produce similar concentration tendencies across the three input representations.

Several lexical and text-based configurations yield lower provisional workload concentration. Average Cosine and Jaccard on titles produces a macro-average of 1.519, a pooled

mean of 1.842, and a Gini coefficient of 0.343. Soft Keyword Coverage on titles produces 1.460, 1.739, and 0.316, respectively. Among the graph-based configurations, Node2Vec on summaries produces a macro-average of 1.497, a pooled mean of 1.755, and a Gini coefficient of 0.299.

Other graph configurations produce substantially more concentrated recommendation lists. For summaries, KG Edge Jaccard produces a macro-average of 2.025, a pooled mean of 2.712, and a Gini coefficient of 0.539. The largest concentration is observed for some local-KG configurations. For example, Local KG (Node2Vec-Cosine) on titles produces a macro-average of 7.114, a pooled mean of 9.338, and a Gini coefficient of 0.680; the corresponding summary values are 6.262, 8.677, and 0.670.

These findings should be interpreted as diagnostics of the ranking outputs rather than measures of final assignment fairness. A method that repeatedly recommends the same reviewers may create a highly concentrated candidate list, but a downstream optimizer can redistribute loads subject to capacity, conflict, and fairness constraints.

Figure 4 complements the scalar summaries by showing Lorenz curves of provisional reviewer-conference recommendation counts. The horizontal axis represents the cumulative proportion of reviewer-conference pairs ordered by increasing recommendation count, and the vertical axis represents their cumulative share of all top-three recommendations. Curves closer to the diagonal correspond to more even recommendation distributions, whereas stronger downward bowing corresponds to greater concentration.

In an operational reviewer-assignment system, the affinity scores evaluated in this section would be supplied to a downstream optimizer. Let $x_{pr} \in \{0, 1\}$ indicate whether reviewer r is assigned to paper p . The optimizer maximizes total affinity while enforcing the required number of reviewers per paper, reviewer lower and upper capacity bounds, and known conflicts of interest. Fairness can be incorporated through workload-balancing penalties, maximum-load constraints, or max-min paper-affinity objectives. Consequently, the

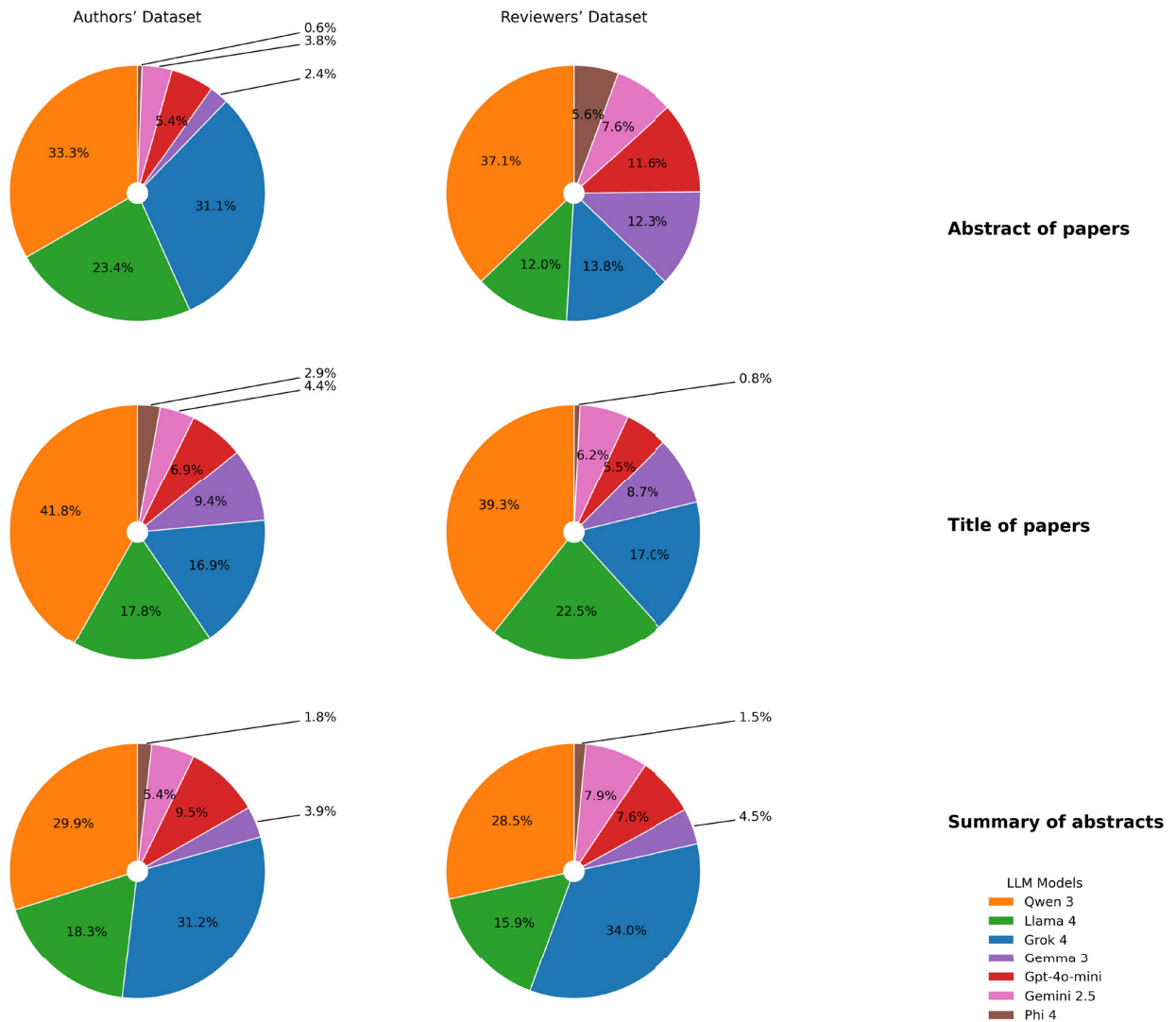


FIGURE 5. Distribution of producer models selected by the judge for titles, abstracts, and summaries in the authors' and reviewers' datasets. Selection frequency describes judge behavior and is not a producer-level ranking-performance comparison.

current ranking and workload analyses evaluate the affinity-construction layer; they do not replace an end-to-end comparison under operational assignment constraints.

Because complete venue-specific capacity and conflict-of-interest records were not available for all conferences, we do not present the current top-three lists as final assignments. Instead, they provide an auditable indication of how each affinity estimator tends to distribute recommendations before constrained optimization.

H. DIAGNOSTIC ANALYSIS OF JUDGE SELECTIONS AND EXTRACTED TOPICS

Figure 5 reports how frequently the judge selected each producer model for the retained keyword profiles. In the authors' dataset, Qwen 3 and Grok 4 account for a large share of the selected outputs, while LLaMA 4 contributes across multiple representations. The reviewer dataset also shows a non-uniform distribution of selected producers.

These distributions describe the selection behavior of the judge under the employed prompts and model versions.

Winner frequency is not a direct measure of downstream reviewer-matching quality. A producer may be selected frequently without necessarily producing rankings that outperform those obtained from another producer. Moreover, the original artifacts retain the winning producer identity and selected keyword list but not all non-selected candidate lists. Therefore, a retrospective single-producer ranking ablation cannot be reconstructed from the stored outputs. Figure 5 should consequently be interpreted as a diagnostic of judge selection rather than as evidence that the ensemble outperforms every individual producer.

Figure 6 presents the 25 most frequent retained keywords for titles, abstracts, and summaries. Across the evaluated representations, terms such as *semantic web*, *knowledge graph*, *linked data*, and *natural language processing* occur frequently, which is consistent with the thematic composition

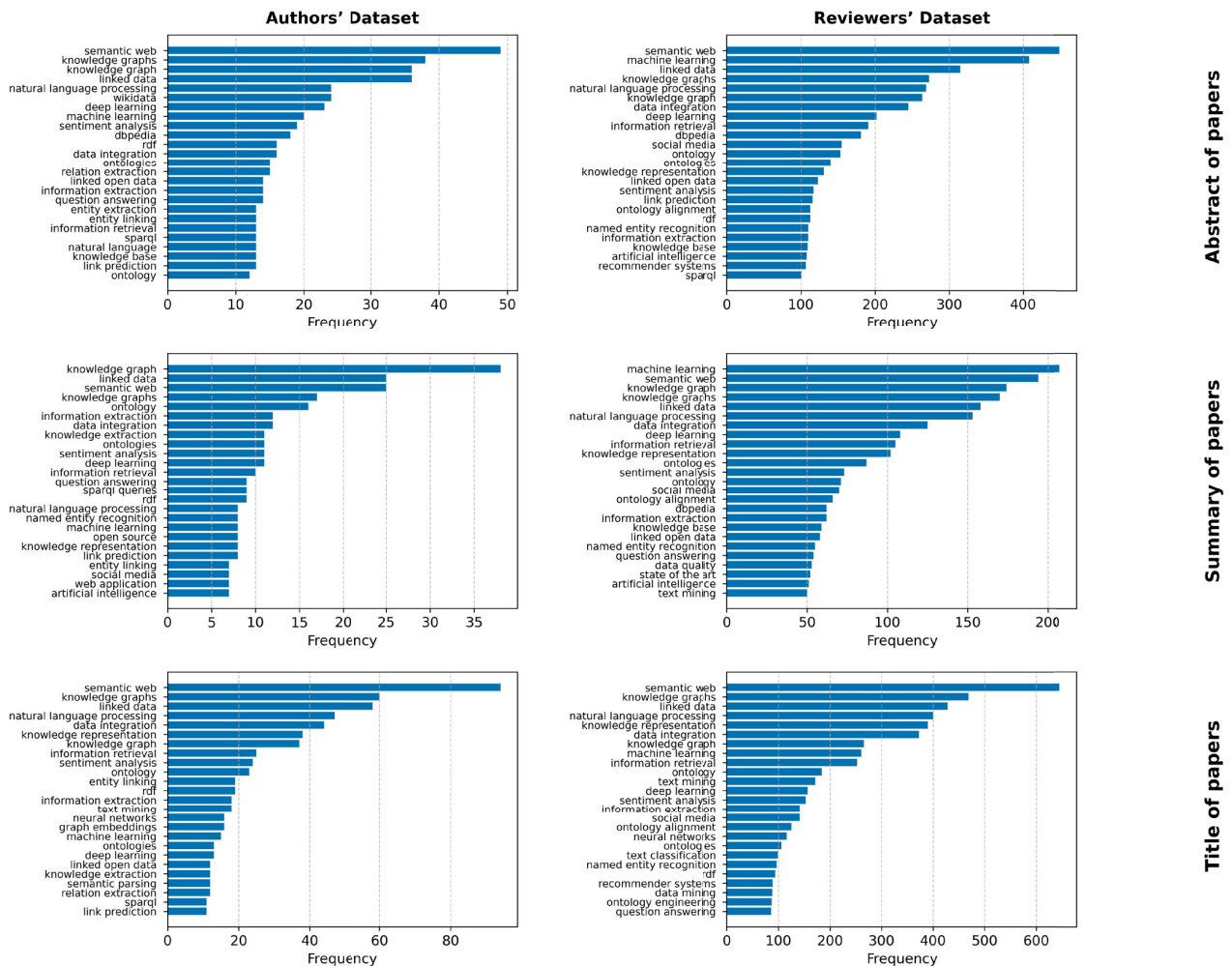


FIGURE 6. Twenty-five most frequent judge-selected keywords from titles, abstracts, and summaries in the authors' and reviewers' datasets.

of the corpus. The reviewer profiles contain a broader vocabulary because they aggregate evidence from multiple prior publications.

The distributions also differ across input representations. Title-based profiles emphasize compact thematic labels, whereas abstract- and summary-based profiles include a larger proportion of methodological phrases. These differences provide descriptive evidence that the retained keywords reflect changes in input length and content granularity.

The frequency analysis demonstrates thematic interpretability, but it does not independently establish extraction accuracy, generation stability, or superiority over alternative keyword-induction methods. Such conclusions require a controlled comparison in which the keyword-induction strategy changes while the downstream matcher and evaluation protocol remain fixed.

VII. CONCLUSION

This paper presented an LLM-assisted, keyword-centric framework for reviewer–paper affinity construction that preserves transparency, auditability, and compatibility with

standard reviewer-assignment pipelines. Rather than using LLMs as autonomous decision-makers, the proposed approach restricts their role to the generation and selection of compact, inspectable keyword profiles. Using a dataset of 663 submissions from 85 computer-science conferences and 524 reviewer profiles built from Semantic Scholar publication summaries, we operationalized an end-to-end workflow that normalizes heterogeneous textual inputs, induces topical keyword sets through a multi-LLM producer ensemble with a judge model, and computes reviewer–paper affinities using complementary lexical, neural, and graph-based matching strategies.

The experimental results show that the most effective matching strategy depends on both the document representation and the desired ranking behavior. For title-based matching, lexical and lexical–overlap fusion methods remain highly competitive, while two-stage bi-encoder/cross-encoder re-ranking provides the strongest first-hit performance. For summary-based matching, keyword-aware semantic retrieval is particularly effective, with Keyword-Aware SBERT achieving the highest P@3 across the evaluated fields.

For abstract-based matching, knowledge-graph structure provides complementary evidence, with KG Edge Jaccard Similarity obtaining the best MRR@3 (0.7843) and MAP@3 (0.2991). These results indicate that reviewer assignment benefits from combining multiple affinity signals rather than relying on a single representation or similarity function.

Beyond ranking quality, the study highlights the practical importance of evaluating reviewer-matching methods in terms of workload realism and assignment-side effects. Several strong text-based baselines yield stable conference-aware workloads with low-to-moderate inequality, while some KG and local-KG variants substantially concentrate recommendations and therefore require careful monitoring before deployment. The winner-model and keyword-frequency analyses further show that the retained keyword profiles are interpretable and domain-consistent within the evaluated dataset, although a small subset of producer models, particularly Qwen 3 and Grok 4, contributes a disproportionate share of the judge-selected keyword sets. Overall, the findings support a pragmatic use of LLMs in reviewer assignment: LLMs can add value when they generate compact and auditable evidence for affinity estimation, while final allocation decisions can remain governed by transparent and configurable assignment components.

Future work should extend the evaluation beyond historical assignments by incorporating expert judgments of reviewer suitability and qualitative assessments of match explanations. Furthermore, sensitivity analyses involving noisy, incomplete, or evolving reviewer profiles would help characterize the robustness of the proposed framework under realistic deployment conditions. In addition, integrating uncertainty-aware ranking and stronger defenses against adversarial text manipulation would better align the system with real-world peer-review threat models. Concurrently, tighter integration with constrained assignment solvers that jointly account for similarity scores, reviewer capacity, conflicts of interest, and fairness objectives could translate the ranking improvements observed in this study into end-to-end gains in final reviewer allocations.

REFERENCES

- [1] J. Jovanovic and E. Bagheri, "Reviewer assignment problem: A scoping review," 2023, *arXiv:2305.07887*.
- [2] X. Zhao and Y. Zhang, "Reviewer assignment algorithms for peer review automation: A survey," *Inf. Process. Manage.*, vol. 59, no. 5, Sep. 2022, Art. no. 103028.
- [3] A. Beygelzimer, Y. N. Dauphin, P. Liang, and J. W. Vaughan, "Has the machine learning review process become more arbitrary as the field has grown? The NeurIPS 2021 consistency experiment," 2023, *arXiv:2306.03262*.
- [4] I. Stelmakh, N. B. Shah, and A. Singh, "PeerReview4All: Fair and accurate reviewer assignment in peer review," in *Proc. Algorithmic Learn. Theory*, 2019, pp. 828–856.
- [5] K. Bouanane, A. N. Medakene, A. Benbelghit, and S. B. Belhaouari, "FairColor: An efficient algorithm for the balanced and fair reviewer assignment problem," *Inf. Process. Manage.*, vol. 61, no. 6, Nov. 2024, Art. no. 103865.
- [6] J. Munkres, "Algorithms for the assignment and transportation problems," *J. Soc. Ind. Appl. Math.*, vol. 5, no. 1, pp. 32–38, 1957.
- [7] L. Charlin and R. Zemel, "The Toronto paper matching system: An automated paper-reviewer assignment system," Univ. Toronto, Toronto, ON, Canada, Tech. Rep., 2013.
- [8] S. Deerwester, S. T. Dumais, G. W. Furnas, T. K. Landauer, and R. Harshman, "Indexing by latent semantic analysis," *J. Amer. Soc. Inf. Sci.*, vol. 41, no. 6, pp. 391–407, 1990.
- [9] D. M. Blei, A. Y. Ng, and M. I. Jordan, "Latent Dirichlet allocation," *J. Mach. Learn. Res.*, vol. 3, pp. 993–1022, Jan. 2003.
- [10] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–10.
- [11] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Human Lang. Technol.*, vol. 1, 2019, pp. 4171–4186.
- [12] I. Beltagy, K. Lo, and A. Cohan, "SciBERT: A pretrained language model for scientific text," 2019, *arXiv:1903.10676*.
- [13] Y. Zhang, Y. Shen, S. Kang, X. Chen, B. Jin, and J. Han, "Chain-of-factors paper-reviewer matching," in *Proc. ACM Web Conf.*, Apr. 2025, pp. 1901–1910.
- [14] Z. Zhuang, J. Chen, H. Xu, Y. Jiang, and J. Lin, "Large language models for automated scholarly paper review: A survey," *Inf. Fusion*, vol. 124, Dec. 2025, Art. no. 103332.
- [15] I. Kuznetsov et al., "What can natural language processing do for peer review?" 2024, *arXiv:2405.06563*.
- [16] X. Gao, J. Ruan, Z. Zhang, J. Gao, T. Liu, and Y. Fu, "ReviewAgents: Bridging the gap between human and AI-generated paper reviews," 2025, *arXiv:2503.08506*.
- [17] T. M. Zhang and N. F. Abernethy, "Reviewing scientific papers for critical problems with reasoning LLMs: Baseline approaches and automatic evaluation," 2025, *arXiv:2505.23824*.
- [18] Y. Zhang, Y. Li, L. Cui, D. Cai, L. Liu, T. Fu, X. Huang, E. Zhao, Y. Zhang, Y. Chen, L. Wang, A. T. Luu, W. Bi, and F. Shi, "Siren's song in the AI ocean: A survey on hallucination in large language models," *Comput. Linguistics*, vol. 2025, pp. 1–46, Jul. 2025.
- [19] S. Farquhar, J. Kossen, L. Kuhn, and Y. Gal, "Detecting hallucinations in large language models using semantic entropy," *Nature*, vol. 630, no. 8017, pp. 625–630, Jun. 2024.
- [20] I. O. Gallegos, R. A. Rossi, J. Barrow, M. M. Tanjim, S. Kim, F. Dernoncourt, T. Yu, R. Zhang, and N. K. Ahmed, "Bias and fairness in large language models: A survey," *Comput. Linguistics*, vol. 50, no. 3, pp. 1097–1179, Sep. 2024.
- [21] T.-L. Lin, W.-C. Chen, T.-F. Hsiao, H.-I. Liu, Y.-H. Yeh, Y. K. Chan, W.-S. Lien, P.-Y. Kuo, P. S. Yu, and H.-H. Shuai, "Breaking the reviewer: Assessing the vulnerability of large language models in automated peer review under textual adversarial attacks," 2025, *arXiv:2506.11113*.
- [22] Z.-Q. Li, H.-L. Xu, H.-J. Cao, Z.-L. Liu, Y.-T. Fei, and J.-P. Liu, "Use of artificial intelligence in peer review among top 100 medical journals," *JAMA Netw. Open*, vol. 7, no. 12, Dec. 2024, Art. no. e2448609.
- [23] L. Charlin, R. S. Zemel, and C. Boutilier, "A framework for optimizing paper matching," *UAI*, vol. 11, pp. 86–95, Jun. 2011.
- [24] B. Li and Y. T. Hou, "The new automated IEEE INFOCOM review assignment system," *IEEE Netw.*, vol. 30, no. 5, pp. 18–24, Sep. 2016.
- [25] X. Liu, T. Suel, and N. Memon, "A robust model for paper reviewer assignment," in *Proc. 8th ACM Conf. Recommender Syst.*, Oct. 2014, pp. 25–32.
- [26] Y. Kalmukov, "Architecture of a conference management system providing advanced paper assignment features," 2011, *arXiv:1111.6934*.
- [27] A. Cohan, S. Feldman, I. Beltagy, D. Downey, and D. S. Weld, "SPECTER: Document-level representation learning using citation-informed transformers," 2020, *arXiv:2004.07180*.
- [28] M. Ostendorf, N. Rethmeier, I. Augenstein, B. Gipp, and G. Rehm, "Neighborhood contrastive learning for scientific document representations with citation embeddings," 2022, *arXiv:2202.06671*.
- [29] C. K. Kreutz and R. Schenkel, "RevASIDE: Assignment of suitable reviewer sets for publications from fixed candidate pools," in *Proc. 23rd Int. Conf. Integr. Web Intell.*, Nov. 2021, pp. 57–68.
- [30] Y. Zhang, H. Cheng, Z. Shen, X. Liu, Y.-Y. Wang, and J. Gao, "Pre-training multi-task contrastive learning models for scientific literature understanding," 2023, *arXiv:2305.14232*.
- [31] A. C. Ribeiro, A. Sizo, and L. P. Reis, "Investigating the reviewer assignment problem: A systematic literature review," *J. Inf. Sci.*, vol. 52, no. 1, pp. 39–59, Feb. 2026.

- [32] M. Aksoy, S. Yanik, and M. F. Amasyali, "Reviewer assignment problem: A systematic review of the literature," *J. Artif. Intell. Res.*, vol. 76, pp. 761–827, Mar. 2023.
- [33] I. Stelmakh, J. Wieting, S. Xi, G. Neubig, and N. B. Shah, "A gold standard dataset for the reviewer assignment problem," 2023, *arXiv:2303.16750*.
- [34] I. Soboroff, "Overview of TREC 2021," *TREC*, 2021.
- [35] A. Kobren, B. Saha, and A. McCallum, "Paper matching with local fairness constraints," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Jul. 2019, pp. 1247–1257.
- [36] C. Cousins, J. Payan, and Y. Zick, "Into the unknown: Assigning reviewers to papers with uncertain affinities," in *Proc. Int. Symp. Algorithmic Game Theory*, 2023, pp. 179–197.
- [37] J. Zhang, H. Zhang, Z. Deng, and D. Roth, "Investigating fairness disparities in peer review: A language model enhanced approach," 2022, *arXiv:2211.06398*.
- [38] T. Eisenhofer, E. Quiring, J. Möller, D. Riepel, T. Holz, and K. Rieck, "No more reviewer# 2: Subverting automatic paper-reviewer assignment using adversarial learning," in *Proc. 32nd USENIX Secur. Symp. (USENIX Secur. 23)*, 2023, pp. 5109–5126.
- [39] J.-Y. Hsieh, "Automated peer-reviewer assignment can be manipulated to secure reviews from colluders," M.S. thesis, Comput. Sci. Dept., School Comput. Sci., Carnegie Mellon Univ., Pittsburgh, PA, USA, May 2024. [Online]. Available: <http://reports-archive.adm.cs.cmu.edu/anon/2024/CMU-CS-24-112.pdf>
- [40] S. Jecmen, N. B. Shah, F. Fang, and L. Akoglu, "On the detection of reviewer-author collusion rings from paper bidding," 2024, *arXiv:2402.07860*.
- [41] R. Zhou, L. Chen, and K. Yu, "Is LLM a reliable reviewer? A comprehensive evaluation of LLM on automatic paper reviewing tasks," in *Proc. Lang. Resour. Eval. Conf.*, May 2024, pp. 9340–9351.
- [42] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, K. Zhu, H. Chen, X. Yi, C. Wang, Y. Wang, W. Ye, Y. Zhang, Y. Chang, P. S. Yu, Q. Yang, and X. Xie, "A survey on evaluation of large language models," *ACM Trans. Intell. Syst. Technol.*, vol. 15, no. 3, pp. 1–45, 2024.
- [43] S. Rose, D. Engel, N. Cramer, and W. Cowley, "Automatic keyword extraction from individual documents," *Text Mining, Appl. Theory*, vol. 2010, pp. 1–20, Jul. 2010.
- [44] R. Mihalcea and P. Tarau, "Textrank: Bringing order into text," in *Proc. Conf. Empirical Methods Natural Lang. Process., Syst. Demonstrations*, 2004, pp. 404–411.
- [45] R. Campos, V. Mangaravite, A. Pasquali, A. M. Jorge, C. Nunes, and A. Jatowt, "YAKE! Collection-independent automatic keyword extractor," in *Proc. Eur. Conf. Inf. Retr.*, 2018, pp. 806–810.
- [46] R. Meng, S. Zhao, S. Han, D. He, P. Brusilovsky, and Y. Chi, "Deep keyphrase generation," 2017, *arXiv:1704.06879*.
- [47] A. Kong, S. Zhao, H. Chen, Q. Li, Y. Qin, R. Sun, and X. Bai, "PromptRank: Unsupervised keyphrase extraction using prompt," 2023, *arXiv:2305.04490*.
- [48] R. Y. Maragheh, C. Fang, C. C. Irugu, P. Parikh, J. Cho, J. Xu, S. Sukumar, M. Patel, E. Korpeoglu, S. Kumar, and K. Achan, "LLM-TAKE: Theme-aware keyword extraction using large language models," in *Proc. IEEE Int. Conf. Big Data (BigData)*, Dec. 2023, pp. 4318–4324.
- [49] N. B. Mansour, H. Rahimi, and M. Alrahabi, "How well do large language models extract keywords? A systematic evaluation on scientific corpora," in *Proc. 1st Workshop AI Scientific Discovery: Directions Opportunities*, 2025, pp. 13–21.
- [50] F. Bagheri, D. Buscaldi, and D. Reforgiato Recupero, "A study on content-based reviewer assignment in the semantic Web and computer science domains," *Computación y Sistemas*, vol. 28, no. 4, pp. 2183–2196, Dec. 2024.
- [51] F. Bagheri, D. Buscaldi, and D. R. Recupero, "Leveraging LLMs for transparent and scalable reviewer-paper matching," *Int. J. Digit. Libraries*, vol. 2026, pp. 349–377, Mar. 2026.
- [52] B. Perozzi, R. Al-Rfou, and S. Skiena, "DeepWalk: Online learning of social representations," in *Proc. 20th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2014, pp. 701–710.
- [53] A. Grover and J. Leskovec, "Node2vec: Scalable feature learning for networks," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2016, pp. 855–864.
- [54] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Inf. Process. Manage.*, vol. 45, no. 4, pp. 427–437, Jul. 2009. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0306457309000259>
- [55] L. Ceriani and P. Verme, "The origins of the Gini index: Extracts from variabilità e mutabilità (1912) by corrado Gini," *J. Econ. Inequality*, vol. 10, no. 3, pp. 421–443, Sep. 2012.



FARID BAGHERI received the Ph.D. degree in computer science from the University of Cagliari, Italy, in 2026. He is currently a Researcher in artificial intelligence with the University of Cagliari and Sorbonne Paris North University, France. His work explores hybrid AI systems that combine symbolic and neural approaches, including knowledge graph-enhanced LLMs, retrieval-augmented generation (RAG), and intelligent agent pipelines. He has contributed to several research studies

in areas, such as reviewer assignment, semantic web applications, and financial risk prediction. His research interests include the integration of large language models and knowledge graphs, with the aim of improving factual accuracy, explainability, and complex information retrieval.



DAVIDE BUSCALDI is currently an Associate Professor with LIPN, Sorbonne Paris North University, France, and a part-time Assistant Professor with École Polytechnique, where he teaches machine learning and data science courses, principally. He has directed or co-directed two Ph.D. theses and is currently directing three more theses in NLP and machine learning. He collaborates in various national and European projects. He is the author of more than 110 peer-reviewed conference papers and journal articles. His main research interests include natural language processing and text mining, in particular the application of modern NLP techniques to text annotation and relation extraction.



DIEGO REFORGIATO RECUPERO was a Post-doctoral Researcher with the University of Maryland, College Park, MD, USA, from 2005 to 2008. Since February 2022, he has been a Full Professor with the Department of Mathematics and Computer Science, University of Cagliari, Italy. He co-founded seven companies in the ICT sector. He is actively involved in European projects and research. He is the author of more than 290 conference papers and journal articles in these research fields, with more than 6000 citations. His current research interests include sentiment analysis, semantic web, natural language processing, human–robot interaction, financial technology, and smart grid.

...